Fleet Management

Software Design Document

Multi-Component Stochastic Degradation MILP

Version 1.0 · October 26, 2026

Contents

1 Project Overview & Goals **3**

2 Architecture **3**
2.1 Package Layout . . . 3
2.2 Data Flow of One Solve . . . 4
2.3 Public API . . . 4
2.4 Cell-Builder Interface . . . 5

3 Index Sets and Notation **5**

4 Input Specification **6**
4.1 Sizes and Horizon . . . 7
4.2 Selectors . . . 7
4.3 Reliability . . . 8
4.4 Repair and Starting Anchors . . . 8
4.5 Initial State . . . 8
4.6 Replacement . . . 8
4.7 Increment Profiles and the Phase Lookup . . . 9
4.8 Model Constants . . . 10
4.9 Costs and Objective . . . 10
4.10 Run Options . . . 11
4.11 Rejected Keys . . . 11
4.12 Consistency Checks . . . 12

5 Decision Variables **14**

6 Objective Function **15**

7 Constraints **15**
7.1 Assignment Constraints . . . 15
7.2 Depot-Day Compatibility . . . 16
7.3 Repair Laws . . . 16
7.4 Encoding of the Dynamics . . . 16
7.5 Replacement . . . 17
7.6 Linear Forms of the Reliability Rows . . . 17
7.7 Gaussian Model . . . 18
7.7.1 Reliability row . . . 19
7.8 Inverse Gaussian Model . . . 19
7.8.1 Reliability row . . . 20
7.9 Wiener Model . . . 21
7.10 Gamma Model . . . 22
7.10.1 Reliability row . . . 23
7.11 Rainflow Model . . . 23
7.11.1 Reliability rows . . . 25
7.12 Repair Cost Variable . . . 26
7.13 Damage Penalty . . . 26
7.14 Loop Constraint . . . 26

8 Full Optimization Problem **27**

9 Output Specification **28**

9.1 Result Dictionary . . . 28
9.2 Rules for the Values . . . 29
9.3 Reports . . . 29
9.4 Result Files . . . 30

10 Horizon Sweeps **30**
10.1 Cost Measures and Ranking . . . 30
10.2 `sweep_operating_horizons` . . . 31
10.3 Early Stop . . . 31
10.4 Report, Checkpoint and Errors . . . 32
10.5 Parallel Cases . . . 32
10.6 `sweep_horizon_grid` . . . 32

11 Warm Starts **33**
11.1 Interface and Reports . . . 33
11.2 Day Mapping . . . 33
11.3 Start, Hints and Completion . . . 34
11.4 Relaxation Start . . . 34
11.5 Warm Starts in Sweeps . . . 34
11.6 Greedy Heuristic . . . 34

12 Plotting API and Layout **35**
12.1 Schedule Plot . . . 35
12.2 Sweep Plot . . . 36

13 Validation Tools **36**
13.1 Structural Validator . . . 36
13.2 Dashboards . . . 37
13.3 Validators of the Gamma Model . . . 37

14 Input File Schema **39**
14.1 Format Rules . . . 39
14.2 HDF5 Layout . . . 40
14.3 Keys by Cell Type . . . 40
14.4 YAML Example (Mixed Models) . . . 41

Project Overview & Goals

The Python package `fleet_management` schedules a fleet of F trains over a horizon of T days. Every day, M missions must be flown. Each train flies at most one mission, spends the day in the depot, or stays idle. Each train has L components, and each component degrades stochastically. On a depot day a component may be repaired (imperfect repair) or replaced. The schedule is a mixed-integer program solved with Gurobi.

Goal. Minimise the cost of depot days, repairs and replacements, plus a penalty on the largest train damage (Section 6), subject to $\Pr(D_{i\ell k} > \tau_{i\ell}) \leq \varepsilon_{i\ell}$ for every component and every day, with $D_{i\ell k}$ the degradation and $\tau_{i\ell}$ the failure threshold. The transitory phase (H_1 days) starts from the current state of the fleet; the operating phase (H_2 days) is the schedule to repeat. The requirement holds on every day of the horizon and of every later repetition of the operating phase (Section 7.14).

Degradation models (input `model`, one per cell):

- **Gaussian**: Gaussian random walk; tracks mean and variance (Section 7.7).
- **Inverse Gaussian (IG)**: tracks the mean; cap on the mean (Section 7.8).
- **Wiener**: Brownian motion with drift; tracks mean and variance; the variance grows by σ^2 per mission day and falls only at a replacement (Section 7.9).
- **Gamma**: Gamma increments with mission-dependent rates; tracks the mean; cap on the mean (Section 7.10).
- **Rainflow**: distribution-free; the increment law enters through summary statistics; one of five tail bounds: Markov, Cantelli, Hoeffding, Bernstein, Chernoff (Section 7.11).

Repair and replacement. Each cell has a repair law (Section 7.3): ARD_∞ (arithmetic reduction of deterioration with infinite memory, the default) or ARD_1 (arithmetic reduction of deterioration of order 1). Every model accepts both laws. Each cell may be replaced (Section 7.5). The loop row forces at least one replacement in the operating phase for a Wiener cell whose train flies a mission in that phase, and for an ARD_1 cell with $\rho_{i\ell} < 1$ that gains mean damage in that phase (scoping document, section "Stochastic loop constraint").

Problem class. A mixed-integer linear program (MILP) when every reliability row is linear and the damage penalty is `inf_norm` or absent ($C_D = 0$); otherwise a mixed-integer quadratically constrained program (MIQCP). It is nonconvex when a quadratic reliability row uses the form `exact` (Section 7.6), and convex when the only quadratic row is the `quadratic` damage penalty (Section 7.13).

Scope of the package. One solve (`solve`); horizon sweeps (Section 10); warm starts (Section 11); the schedule plot (Section 12); validation tools (Section 13). Inputs in YAML, JSON or HDF5 (Section 14); outputs in Section 9.

Authority. This specification is authoritative for the formulation and for the tools. Every change is written here first; the code follows it.

Non-goals.

- Routing decisions are not optimised: the missions are inputs.
- Continuous-time formulations are out of scope: one day is the time unit.

Architecture

Package Layout

```
Fleet_Management/
  pyproject.toml              Python >= 3.10; extra "dashboard"
```

```
README.md
spec/                        this specification: spec.tex, references.bib
examples/                    example inputs and regression scripts
tests/                       pytest suite
experiments/                 experiments: inputs, runners, job scripts
results/validator_tests/     fixtures of the validator tests (test data)
euler/                       two Slurm jobs of the ETH Euler cluster
src/fleet_management/
  __init__.py                public API
  config.py                  reader, checks, broadcasting; load_config
  solver.py                  solve(): checks, dispatch, result files
  horizon_sweep.py           sweep_operating_horizons, sweep_horizon_grid
  greedy_warm_start.py       build_greedy_warm_start
  validator_dashboard.py     Streamlit entry point of the dashboards
  degradation_model/
    base.py                  shared model, registry, objective,
                             warm starts, result extraction,
                             performance counters
    gaussian.py, inverse_gaussian.py, wiener.py, gamma.py, rainflow.py
                             cell builders; gamma.py computes the cap A_1
    gamma_utils/
      gamma_replay_validator.py      replay validator
      gamma_stochastic_validator.py  Monte Carlo validator
      gamma_tail_validator.py        exact validator
      gamma_tail_bound.py            moschopoulos_tail_probability
  utils/plotter.py           plot_management, plot_horizon_sweep
  validation/
    validator.py             validate
    dashboard_inspector.py, dashboard_gamma.py
```

`examples/regression/` holds the scripts of the two Slurm jobs (Slurm is the job scheduler of the cluster): the job `gamma_horizon_convergence.sbatch` runs `run_horizon_sweep.py`, and the job `vbz_progressive_feasibility.sbatch` runs `run_progressive_vbz_feasibility.py`.

Data Flow of One Solve

```
solve(input_path, results_path="output.yaml", *,
      warm_start=None, warm_start_mode="start") -> dict
```

1. Check the input file: it exists (`FileNotFoundError`) and has the extension `.yaml`, `.yml`, `.json`, `.h5` or `.hdf5` (`ValueError`). Check `results_path` (Section 9.4).
2. Read the file with the input reader (Section 14) into a mapping.
3. `load_config`: normalise the mapping into a `FleetConfig` and run the checks of Section 4.12 (`ValueError`).
4. Build the model (Section 2.4), apply the warm start (Section 11) and optimise.
5. Extract the result (Section 9): the shared extraction, then the `extract` hook of each builder. Write the result file (Section 9.4) and return the result.

Steps 3 to 5 form the internal entry point `solve_mapping(data, results_path=None, *, warm_start=None, warm_start_mode="start")`, which takes the mapping of step 2. Gurobi errors are not caught.

Public API

The package exports:

- one solve: `solve` (Section 2.2);

- horizon sweeps: `sweep_operating_horizons` and `sweep_horizon_grid` (Section 10);
- plots: `plot_management` and `plot_horizon_sweep` (Section 12);
- the structural validator `validate` (Section 13.1);
- the Gamma validators (Section 13.3): the replay validator `validate_gamma_replay_schedule` with its file version `validate_gamma_replay_files`; the Monte Carlo validator `validate_gamma_stochastic_schedule`; and the selector of both, `validate_gamma_schedule`, with its file version `validate_gamma_schedule_files`.

Internal functions, not exported: `solve_mapping` and `load_config` (Section 2.2); the greedy heuristic `build_greedy_warm_start` (Section 11.6); the exact Gamma validator `validate_gamma_tail_bound_schedule` and its file version (Section 13.3).

Cell-Builder Interface

A registry maps each model name (`gaussian`, `inverse_gaussian`, `wiener`, `gamma`, `rainflow`) to a builder; the model of cell (i, ℓ) is its entry of `model`.

```
register_cell_builder(name, builder)  # CELL_BUILDERS[name] = builder
get_cell_builder(name)     # NotImplementedError, with the names, if
    absent
dispatch_cell(ctx, i, l)   # get_cell_builder(ctx.model_of[i, l])
                           #     .add_cell(ctx, i, l)
```

A builder has the attribute `name` and three hooks:

- `prepare(ctx, cfg, cells, opts)`: called once per model in the fleet, with its cells. It stores the per-cell arrays of the model in `ctx`, creates the variables of the tracked quantities, anchors and selectors of these cells only, may tighten variable bounds, and sets Gurobi's `NonConvex` to 2 if some cell keeps an exact quadratic row.
- `add_cell(ctx, i, l)`: adds the rows of one cell (Section 7): dynamics in the form of `encoding`, replacement and anchor rows, repair-cost rows, reliability rows in the form of `reliability_impl`, loop rows of the full tracked state.
- `extract(ctx, cfg, out)`, optional: adds model-specific entries to the result `out` after the shared entries.

The context `ctx` holds the sizes F, L, M, H_1, H_2, T; the Gurobi model; the shared variables x, x^m, x^r (absent if replacement is not allowed), μ, z (cells with $C_R > 0$) and u (if $C_D > 0$), with the shapes of Section 5; the per-cell arrays (model, τ, ε, ρ, initial and replacement values, starting anchors); the increment accessor $(i, j, \ell, k) \mapsto$ increment after the phase lookup, Eq. (1), with $j = 0$ the depot; the run options; a dictionary `extras`.

Building order:

1. Create the model, apply the run options, create the shared variables.
2. Call `prepare` for each model present, in alphabetical order of the names.
3. Build the objective (Section 6) and the rows that do not depend on the model: assignment (Section 7.1), depot day (Section 7.2), damage penalty (Section 7.13).
4. Call `add_cell` for every cell, i outer, ℓ inner.

Each builder defines $\mu_{i\ell k}$, $k = 1, \ldots, T$, for its cells. A model is added by a module with its builder, its registration, its input checks in `config.py` (Section 4.12) and its section in this specification.

Index Sets and Notation

Symbol	Meaning	Defined in
Indices and horizon		
$i \in \{1, \dots, F\}$	train	
$\ell \in \{1, \dots, L\}$	component of a train	
(i, ℓ)	a *cell*: component ℓ of train i	
$j \in \{0, 1, \dots, M\}$	activity: $j = 0$ is a depot day, $j \geq 1$ a mission	
$k \in \{1, \dots, T\}$	day; index $k = 0$ holds the initial state	
$H = [H_1, H_2]$, $T = H_1 + H_2$	horizon input; a scalar H means $H_1 = H_2 = H$	4.1
days $1, \dots, H_1$	transitory phase	
days $H_1 + 1, \dots, T$	operating phase	7.14
$p \in \{1, \dots, P\}$	segment of the `pwl` form, $P =$ `pwl_points`	7.6
Parameters, increments and states		
$\tau_{i\ell}$, $\varepsilon_{i\ell}$, $\rho_{i\ell}$	failure threshold, reliability level, repair efficiency of a cell; a model section may drop the subscript $i\ell$ after saying so	4
C_M; $C_{R,i\ell}$, $C_{\mathrm{rep},i\ell}$; C_D	costs: depot day; repair, replacement; damage penalty	6
$\mu^{\mathrm{inc}}_{i\ell jk}$, $v^{\mathrm{inc}}_{i\ell jk}$, $b^{\mathrm{inc}}_{i\ell jk}$, $K^{\mathrm{inc}}_{i\ell jk}$, $\beta_{i\ell jk}$	mean, variance, support width and cumulant generating function (CGF) of the increment of cell (i, ℓ) when train i flies mission j on day k, after the phase lookup; Gamma rate	4.7
$\delta\mu_{i\ell k} = \sum_{j=1}^{M} x_{ijk}\, \mu^{\mathrm{inc}}_{i\ell jk}$	day increment of the mean; δv, δR (from $(b^{\mathrm{inc}})^2$) and δK likewise; 0 on depot and idle days	
$\mu_{i\ell k}$, $v_{i\ell k}$, $R_{i\ell k}$, $K_{i\ell k}$	tracked mean, variance, Hoeffding budget, Chernoff quantity	5
$g^{\mu}_{i\ell k}$, $g^{v}_{i\ell k}$, $g^{R}_{i\ell k}$, $g^{K}_{i\ell k}$	anchors (law ARD_1)	7.3
$\mu_{0,i\ell}$, $v_{0,i\ell}$, $R_{0,i\ell}$, $K_{0,i\ell}$	initial values	4
$\mu^{\mathrm{new}}_{i\ell}$, $v^{\mathrm{new}}_{i\ell}$, $R^{\mathrm{new}}_{i\ell}$, $K^{\mathrm{new}}_{i\ell}$	replacement values	7.5
$b^{0}_{i\ell}$, $b^{\mathrm{new}}_{i\ell}$	support widths of the initial and replacement damage	7.11
ARD_∞, ARD_1; a; ψ	repair laws; repair factor; repair map	7.3
Derived constants		
M_s	natural range of a tracked quantity s (big-M constant)	7.4
Φ	standard normal distribution function	7.7
$\kappa_G = \Phi^{-1}(1 - \varepsilon)$	standard normal quantile	7.7
$V_{\max} = \tau^2/\kappa_G^2$	variance range, Gaussian and Wiener	7.7
$\eta_{i\ell}$; $Z \sim \mathrm{IG}(1, 1/\eta)$; q; $\bar{\mu} = \tau/\mathbb{E}[Z \mid Z > q]$	inverse Gaussian dispersion; unit law; its $(1 - \varepsilon)$-quantile; mean cap	7.8
$\beta^{*}_{i\ell}$; $t = \beta^{*}\tau$; $A_1(t, \varepsilon)$; $Q(a, x)$	smallest Gamma rate among the terms of the cell with a positive mean; scaled threshold; shape cap; regularised upper incomplete gamma function	7.10
$\kappa_C = \sqrt{(1 - \varepsilon)/\varepsilon}$	Cantelli coefficient	7.11
$\kappa = \ln(1/\varepsilon)$	exponent constant (Hoeffding, Bernstein)	7.11
$b_{i\ell}$, $c_{i\ell} = \kappa\, b_{i\ell}/3$	Bernstein support constant and margin	7.11
$s_{i\ell}$	Chernoff parameter (input `s_chernoff`)	7.11
V^{C}, V^{B}	variance ranges, rainflow Cantelli and Bernstein	7.11
$\mu_r =$ `tangent_ref` $\cdot\, \tau$; c_2, c_1	tangent point; constants of a quadratic row	7.6
E	evaluation horizon (input `evaluation_horizon`)	6

Input Specification

An input is one mapping from key names to values. A *cell* is a pair (i, ℓ) of a train and a component. The column *Applies to* uses the abbreviations G (Gaussian), IG (inverse Gaussian), W (Wiener), Ga (Gamma), RF (rainflow) and, for a rainflow cell, Mk (Markov), Ca (Cantelli), Ho (Hoeffding), Be (Bernstein), Ch (Chernoff); *all* is every cell. A *cell value* is a scalar, a list of L numbers (one per component, copied to every train) or an $F \times L$ array; a scalar is copied to every cell. A *selector* has the same shapes with strings. A *profile* is defined in Section 4.7; a *flag*

is `true` or `false`. No other broadcasting is accepted.

At a cell where a key does not apply, an array entry may be `null` and the key may be omitted; where it applies, `null` is an error. A broadcast value (a scalar, a list, a profile without the axis F) is read only where the key applies. A non-`null` entry of an array with one entry per cell ($F \times L$ array, or profile with the axes F and L) at a cell where the key does not apply is ignored with a `UserWarning`; a selector entry there is still checked (Section 4.12). Key names are case sensitive; a key outside the tables raises an error (Section 4.11).

Sizes and Horizon

Key	Shape	Default	Applies to	Meaning
`F`	integer	required	all	Number of trains; $F > M$
`M`	integer	required	all	Number of missions
`L`	integer	inferred	all	Components per train; the length of the component axis of `model`. Required if `model` is one string and $L > 1$
`H`	integer or `[H1, H2]`	required	all	Horizon; a scalar H means $H_1 = H_2 = H$

The horizon has $T = H_1 + H_2$ days: days $1, \ldots, H_1$ form the transitory phase, days $H_1 + 1, \ldots, T$ the operating phase. A profile has H_2 entries for the operating phase and, optionally, H_1 entries for the transitory phase.

Selectors

Key	Shape	Default	Applies to	Meaning
`model`	selector	required	all	`gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`
`repair_model`	selector	`ardinf`	all	Repair law (Section 7.3): `ardinf` or `ard1`; every model accepts both
`bound_method`	selector	`cantelli`	RF	Tail bound (Section 7.11): `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`
`reliability_impl`	selector	`exact`	G, W, RF (Ca, Be, Ho)	Form of the reliability row (Section 7.6): `exact`, `tangent` or `pwl`. A cell with a linear row (IG, Ga, Mk, Ch) is built in the form `exact`; the value is ignored
`encoding`	selector	`bigm`	all	Encoding of the dynamics rows (Section 7.4): `bigm` or `indicator`
`component_names`	list of L strings	`Component` n for component n	all	Name of each component, used by the plots

Reliability

Key	Shape	Default	Applies to	Meaning
`tau`	cell value	required	all	Degradation threshold $\tau_{i\ell}$
`epsilon`	cell value	required	all	Reliability level $\varepsilon_{i\ell}$: bound on $\Pr(D > \tau_{i\ell})$ on every day
`tangent_ref`	cell value	0.5	cells with form `tangent`	Tangent point $\mu_r = \texttt{tangent_ref} \cdot \tau$
`pwl_points`	integer	8	cells with form `pwl`	Number P of segments

Repair and Starting Anchors

Key	Shape	Default	Applies to	Meaning
`rho`	cell value	required	all	Repair efficiency $\rho_{i\ell}$
`anchor_mu_0`	cell value	μ_0	`ard1` cells	Starting anchor $g^{\mu}_{i\ell 0}$ of the mean
`anchor_v_0`	cell value	v_0	`ard1` cells at G, RF (Ca, Be)	Starting anchor $g^{v}_{i\ell 0}$ of the variance
`anchor_R_0`	cell value	R_0	`ard1` cells at RF (Ho)	Starting anchor $g^{R}_{i\ell 0}$ of the Hoeffding budget
`anchor_K_0`	cell value	K_0	`ard1` cells at RF (Ch)	Starting anchor $g^{K}_{i\ell 0}$ of the Chernoff quantity

Initial State

Key	Shape	Default	Applies to	Meaning
`mu_0`	cell value	required	all	Initial mean degradation $\mu_{0,i\ell}$
`v_0`	cell value	0	G, W, RF (Ca, Be)	Initial variance $v_{0,i\ell}$
`support_0`	cell value	0	RF (Be)	Width $b^0_{i\ell}$ of the range of the initial damage
`R_0`	cell value	0	RF (Ho)	Initial Hoeffding budget $R_{0,i\ell}$: the square of the width of the range of the initial damage
`cgf_0`	cell value	$s\,\mu_0$	RF (Ch)	Initial Chernoff quantity $K_{0,i\ell}$: the cumulant generating function of the initial damage at $s_{i\ell}$

Replacement

Key	Shape	Default	Applies to	Meaning
`replacement_mu`	cell value	0	all	Mean $\mu^{\text{new}}_{i\ell}$ after a replacement
`replacement_v`	cell value	0	G, W, RF (Ca, Be)	Variance $v^{\text{new}}_{i\ell}$ after a replacement

Key	Shape	Default	Applies to	Meaning
`support_new`	cell value	0	RF (Be)	Width $b^{\text{new}}_{i\ell}$ of the range of the damage after a replacement
`R_new`	cell value	0	RF (Ho)	Hoeffding budget $R^{\text{new}}_{i\ell}$ after a replacement
`cgf_new`	cell value	$s\,\mu^{\text{new}}$	RF (Ch)	Chernoff quantity $K^{\text{new}}_{i\ell}$ after a replacement
`replacement_as_new`	flag	`false`	all	If `true`, every replacement input above must be 0 (the replaced part is in the zero state)
`allow_replacement`	flag	`true`	all	If `false`, the variables x^r and their rows are not built

Increment Profiles and the Phase Lookup

A profile gives a parameter of the damage increment when train i flies mission $j \in \{1, \dots, M\}$, for component ℓ. Its canonical shape is (F, L, M, H_*), with $H_* = H_2$ for an operating profile and $H_* = H_1$ for a transitory profile. The parser accepts the shapes scalar, (M), (L, M), (L, M, H_*), (F, L, M) and (F, L, M, H_*), and expands each of them to (F, L, M, H_*) before any row is built (eager broadcasting); a missing axis is repeated. An array whose shape fits both (L, M, H_*) and (F, L, M) is read as (L, M, H_*). `gamma_beta` and `gamma_beta_trans` read a list as one rate per component and a two-dimensional array as one rate per cell; rates per mission have the shape (F, L, M).

Let p be an operating profile and p^{tr} the transitory profile of the same key, if given. The value used on day $k \in \{1, \dots, T\}$ is

$$p^{\text{inc}}_{i\ell jk} = \begin{cases} p^{\text{tr}}_{i\ell jk} & k \le H_1 \text{ and } p^{\text{tr}} \text{ is given,} \\ p_{i\ell j,\,((k-1) \bmod H_2)+1} & k \le H_1 \text{ and } p^{\text{tr}} \text{ is not given,} \\ p_{i\ell j,\,((k-H_1-1) \bmod H_2)+1} & k > H_1. \end{cases} \tag{1}$$

Each profile key is looked up on its own. The symbols $\mu^{\text{inc}}_{i\ell jk}$, $v^{\text{inc}}_{i\ell jk}$, $b^{\text{inc}}_{i\ell jk}$ and $K^{\text{inc}}_{i\ell jk}$ denote the values after the lookup.

Key	Shape	Default	Applies to	Meaning
`mu`	profile	required	all	Mean $\mu^{\text{inc}}_{i\ell jk}$ of the increment; for Ga the increment has shape $\mu\,\beta$ and rate β
`v`	profile	required	G, RF (Ca, Be)	Variance $v^{\text{inc}}_{i\ell jk}$ of the increment
`support`	profile	required	RF (Ho, Be)	Width $b^{\text{inc}}_{i\ell jk}$ of the range of the increment
`cgf`	profile	required	RF (Ch)	Cumulant generating function $K^{\text{inc}}_{i\ell jk} = \ln \mathbb{E}\, e^{sX}$ of the increment X at $s = s_{i\ell}$
`s_chernoff`	cell value	required	RF (Ch)	Chernoff parameter $s_{i\ell}$
`gamma_beta`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_2)	required	Ga	Rate $\beta_{i\ell jk}$ of the Gamma increment; a shape without a day axis is repeated along the days

Key	Shape	Default	Applies to	Meaning
`mu_trans`, `v_trans`, `support_trans`, `cgf_trans`	transitory profile	none	as the key without the suffix	Transitory-phase profile of the key; it requires the operating profile
`gamma_beta_trans`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_1)	none	Ga	Rates of the transitory phase

Model Constants

Key	Shape	Default	Applies to	Meaning
`sigma`	cell value	required	W	Diffusion constant $\sigma_{i\ell}$: every mission adds $\sigma_{i\ell}^2$ to the variance
`eta`	cell value	required	IG	Dispersion $\eta_{i\ell}$ of the inverse Gaussian law of each term (Section 7.8)
`v_max_user`	cell value	none	G, W, RF (Ca, Be)	Upper bound $V_{\max,i\ell}^{\text{user}}$ on the variance; if given, the row $v_{i\ell k} \le V_{\max,i\ell}^{\text{user}}$ holds on every day
`gamma_beta_0`	cell value	none (known damage)	Ga	Rate of the initial damage, a Gamma variable with mean μ_0; if omitted, the initial damage is the known constant μ_0 and does not enter the smallest rate
`gamma_beta_new`	cell value	none (known damage)	Ga	Rate of the damage after a replacement, a Gamma variable with mean μ^{new}; if omitted, it is the known constant μ^{new} and does not enter the smallest rate

Costs and Objective

Key	Shape	Default	Applies to	Meaning
`C_M`	scalar	required	all	Cost of one depot day of one train, whatever the number of jobs on that day
`C_R`	cell value	required	all	Repair cost $C_{R,i\ell}$ per unit of degradation removed; at a cell with $C_{R,i\ell} = 0$ the variable z and its rows are not built
`C_rep`	cell value	required	all	Replacement cost $C_{\text{rep},i\ell}$
`C_D`	scalar	required	all	Damage penalty coefficient C_D; if $C_D = 0$, the variable u and its rows are not built and `penalty_type` is ignored

Key	Shape	Default	Applies to	Meaning
`penalty_type`	string	`inf_norm`	all	Damage penalty (Section 7.13): `inf_norm` or `quadratic`; if $C_D > 0$, `quadratic` requires `reliability_impl exact` at every cell with a quadratic reliability row (G, W, RF with Ca, Ho or Be); another value is an error

Run Options

Key	Shape	Default	Meaning
`verbose`	integer	1	Gurobi output flag: 0 or 1
`mip_gap`	number	0.1	Relative MIP gap at which Gurobi stops
`time_limit`	number	3600	Time limit of Gurobi, in seconds
`fast`	flag	`false`	If `true`: `MIPFocus` = 1, `Heuristics` = 0.5, `ImproveStartGap` = 0.5, `NoRelHeurTime` = max(2, 0.15 `time_limit`)
`gurobi_params`	mapping	none	Gurobi parameters by name, set after every other option
`objective_mode`	string	`total`	`total`, `operating_average` or `evaluation_total` (Section 6)
`evaluation_horizon`	integer	none	Evaluation horizon E; required if `objective_mode` is `evaluation_total`
`progress_interval_seconds`	number	5.0	Minimum time between two records of the progress report (Section 9.3)
`relaxation_warm_start`	flag	`false`	If `true` and no other warm start is applied, a relaxation start is computed (Section 11)
`relaxation_time_limit`	number	60.0	Time limit of the relaxation start, in seconds
`validation`	mapping	none	Settings of the Gamma validators (Section 13.3): `mode`, `repetitions`, `random_seed`, `maximum_schedule_failure_rate`, `confidence_level`, `batch_size`; `solve` does not read it

No input key selects a warm start from a previous result (Section 11).

Rejected Keys

A key of the table raises an error whose message names the canonical key and, where the meaning differs, the meaning of the canonical key. Any other key outside the tables raises the error "unknown key" with the list of the canonical keys. The check on keys runs before any array is read.

Rejected key	Message: canonical key and meaning
`maintenance_type`	Use `repair_model`. Its values are `ardinf` (the repair reduces the whole level) and `ard1` (it reduces the level accumulated since the last intervention). In a file written with `maintenance_type`, the value `ARD1` corresponds to `ardinf`, and the value `ARA1` to `ard1`
`tail_bound`, `method`	Use `bound_method`; its values are `markov`, `cantelli`, `hoeffding`, `bernstein` and `chernoff`
`formulation`	Use `reliability_impl`. The value `exact` keeps its meaning. The value `lp` corresponds to `tangent` with `tangent_ref` equal to 0
`beta`	Use `gamma_beta`. It is a rate, and `beta` is a scale: `gamma_beta` equals 1/`beta`
`alpha_inc`	The Gamma increments are given by the mean `mu` and the rate `gamma_beta`. The shape of an increment is `mu` times `gamma_beta`
`gamma_beta_bound`	Remove it. The formulation computes the smallest rate of each cell from the rates of the cell (Section 7.10)
`gamma_calibration_method`	Remove it. The formulation has no calibration step (Section 7.10)
`b_inc`	Use `support`: the width of the range of the increment
`b_0`	Use `support_0`: the width of the range of the initial damage
`b_new`	Use `support_new`: the width of the range of the damage after a replacement
`n_workers`, `warm_start`	Remove them. Parallel runs and warm starts between horizons are options of the sweep functions (Sections 10 and 11)
`depot_capacity`	Remove it. The availability of the depot is not limited at fleet level (Section 7.1)
`C_P`	Remove it. The loop constraint is a hard constraint and has no penalty (Section 7.14)
`C_S`	Use `C_D`, with the same meaning
`alpha`	Use `tau`, with the same meaning
`xi`, `repair_rho`	Use `rho`, with the same meaning
`mu_new`	Use `replacement_mu`, with the same meaning
`v_new`	Use `replacement_v`, with the same meaning

Consistency Checks

The solver validates all of the following before it builds the model. A violation raises a `ValueError` whose message names the key and, for a cell value, the cell (i, ℓ). A check of one model runs only at the cells of that model, on the entries where the key applies.

Keys, shapes and entries.

- Every key is in the tables of Section 4. The keys `F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep`, `C_D` and the keys that each cell type requires (Section 14) are present.
- Each value has one of the shapes of its table row; the last axis of an operating profile has H_2 entries and that of a transitory profile H_1; `component_names` has L entries.
- At a cell where a key applies, every entry is a finite number (a string for a selector); `null` is an error.

Sizes and selectors.

- F, M and L are positive integers, and $F > M$. A given `L` equals the length of the component axis of `model` when `model` is a list or an array.
- Every entry of `component_names` is a string that is not empty after blanks are removed.
- H is an integer ≥ 1, or a pair of integers $H_1, H_2 \geq 1$.

- `model` entries are `gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`; `repair_model` entries are `ard1` or `ardinf`, for every model.
- `bound_method` entries at rainflow cells are `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`.
- Every non-`null` entry of `reliability_impl` is `exact`, `tangent` or `pwl`, also at a cell whose row is linear (IG, Ga, Mk, Ch); there the value is ignored and the form `exact` is built. `encoding` entries are `bigm` or `indicator`.
- `penalty_type` is `inf_norm` or `quadratic`. If $C_D > 0$ and `penalty_type` is `quadratic`, `reliability_impl` is `exact` at every cell with a quadratic reliability row (G, W, and RF with Ca, Ho or Be); another value is an error.

Reliability, repair and initial state.

- $\tau_{i\ell} > 0$ and finite.
- $\varepsilon_{i\ell} \in (0, 1)$, and $\varepsilon_{i\ell} < 0.5$ at G and W cells.
- $\rho_{i\ell} \in (0, 1]$.
- $0 \leq \mu_{0,i\ell} < \tau_{i\ell}$.
- v_0, `support_0` and R_0 are ≥ 0; if `allow_replacement` is `true`, so are `replacement_mu`, `replacement_v`, `support_new` and R^{new}.
- `tangent_ref` $\in [0, 1)$ at cells with the form `tangent`; `pwl_points` is an integer ≥ 1.
- Starting anchors: $0 \leq$ `anchor_mu_0` $\leq \mu_0$, $0 \leq$ `anchor_v_0` $\leq v_0$, $0 \leq$ `anchor_R_0` $\leq R_0$ and $0 \leq$ `anchor_K_0` $\leq K_0$, at the cells that read them.
- If `replacement_as_new` is `true`, `replacement_mu`, `replacement_v`, `support_new`, `R_new` and `cgf_new` are 0 at every cell that reads them; the check does not run when the flag is `false`.

Natural ranges and replacement states.

- Every initial value (μ_0, v_0, R_0, K_0) and every replacement value (μ^{new}, v^{new}, R^{new}, K^{new}; only if `allow_replacement` is `true`) lies in $[0, M_s]$, with the natural range M_s of its state: τ for a mean; $V_{\max} = \tau^2/\big(\Phi^{-1}(1-\varepsilon)\big)^2$ for the variance at G and W cells; $\varepsilon\tau^2/(1-\varepsilon)$ at Ca cells; $\tau(\tau - 2c)/(2\kappa)$ at Be cells, with $c = \kappa b/3$ and $\kappa = \ln(1/\varepsilon)$; $2\tau^2/\kappa$ for R at Ho cells; $s\tau - \kappa$ for K at Ch cells (Sections 7.7 to 7.11). Hence every starting anchor lies in its range.
- If `allow_replacement` is `true`, the replacement state $(\mu^{\text{new}}, v^{\text{new}}, R^{\text{new}}, K^{\text{new}})$ of every cell satisfies every row that holds on a replacement day, in the form built at the cell: the reliability row in the form of `reliability_impl` (for `pwl`, the cap of the segment that contains μ^{new}), its side row, and $v^{\text{new}} \leq V^{\text{user}}_{\max,i\ell}$ where `v_max_user` is given (Sections 7.7 to 7.11 and 7.6).

Increments.

- $0 \leq \mu^{\text{inc}}_{i\ell jk} < \tau_{i\ell}$ for every mission j and every day $k \in \{1, \ldots, T\}$.
- Where $\mu^{\text{inc}}_{i\ell jk} = 0$, the values $v^{\text{inc}}_{i\ell jk}$, $b^{\text{inc}}_{i\ell jk}$ and $K^{\text{inc}}_{i\ell jk}$ are 0.
- $v^{\text{inc}}_{i\ell jk}$, $b^{\text{inc}}_{i\ell jk}$ and $K^{\text{inc}}_{i\ell jk}$ are ≥ 0. A transitory profile has its operating profile.

Gaussian, inverse Gaussian, Wiener and Gamma cells.

- G: $\mu_0 \geq 3\sqrt{v_0}$, $\mu^{\text{inc}}_{i\ell jk} \geq 3\sqrt{v^{\text{inc}}_{i\ell jk}}$ for every mission j and day k, and, if `allow_replacement` is `true`, $\mu^{\text{new}} \geq 3\sqrt{v^{\text{new}}}$.
- IG: $\eta_{i\ell} > 0$.

- W: $\sigma_{i\ell} > 0$ and $v_{0,i\ell} \geq 0$.
- Ga: every entry of `gamma_beta` and `gamma_beta_trans` is finite and > 0; `gamma_beta_0` and `gamma_beta_new`, where given, are finite and > 0.
- G, W, RF (Ca, Be) with `v_max_user` given: $V^{\text{user}}_{\max,i\ell} > 0$, $V^{\text{user}}_{\max,i\ell} \geq v_{0,i\ell}$ and, if `allow_replacement` is `true`, $V^{\text{user}}_{\max,i\ell} \geq v^{\text{new}}_{i\ell}$.

Rainflow cells.

- Be: $v^{\text{inc}}_{i\ell jk} \leq (b^{\text{inc}}_{i\ell jk})^2/4$ for every mission j and day k, $v_0 \leq (b^0)^2/4$ and, if `allow_replacement` is `true`, $v^{\text{new}} \leq (b^{\text{new}})^2/4$.
- Be: $\tau_{i\ell} > 2\kappa\, b_{i\ell}/3$, with $\kappa = \ln(1/\varepsilon_{i\ell})$ and $b_{i\ell}$ the support constant of Section 7.11.
- Be with the form `tangent`: `tangent_ref` $\cdot\, \tau_{i\ell} < \sqrt{\tau_{i\ell}\,(\tau_{i\ell} - 2c_{i\ell})}$, with $c_{i\ell} = \kappa\, b_{i\ell}/3$; Be with the form `pwl`: $\tau_{i\ell}/(2P) < \sqrt{\tau_{i\ell}\,(\tau_{i\ell} - 2c_{i\ell})}$, with $P =$ `pwl_points`.
- Ch: `s_chernoff` is given and $s_{i\ell} > \ln(1/\varepsilon_{i\ell})/\tau_{i\ell}$.
- Ch: $K^{\text{inc}}_{i\ell jk} \geq s_{i\ell}\, \mu^{\text{inc}}_{i\ell jk}$ for every mission j and day k, $K_0 \geq s\,\mu_0$, at `ard1` cells $g^K_0 \geq s\, g^{\mu}_0$, and, if `allow_replacement` is `true`, $K^{\text{new}} \geq s\,\mu^{\text{new}}$.

Run options and costs.

- `objective_mode` is `total`, `operating_average` or `evaluation_total`; in the last mode `evaluation_horizon` is given. If `evaluation_horizon` is given, in any mode, it is an integer $E > H_1$.
- `mip_gap` ≥ 0; `time_limit`, `progress_interval_seconds` and `relaxation_time_limit` are > 0; `gurobi_params` and `validation` are mappings; `fast`, `relaxation_warm_start`, `allow_replacement` and `replacement_as_new` are flags.
- C_M, C_D, $C_{R,i\ell}$ and $C_{\text{rep},i\ell}$ are finite and ≥ 0.

Decision Variables

A variable and its rows are built only where a row or an objective term with a non-zero coefficient uses them. The day index runs over $k = 1, \ldots, T$; index $k = 0$ holds constants (initial values and starting anchors).

Variable	Shape, type	Built when	Meaning
x_{ijk}	$F \times (M{+}1) \times T$, binary	always	1 if train i flies mission j on day k; $j = 0$: depot day
$x^m_{i\ell k}$	$F \times L \times T$, binary	always	1 if component ℓ of train i is repaired on day k
$x^r_{i\ell k}$	$F \times L \times T$, binary	`allow_replacement = true`	1 if component ℓ of train i is replaced on day k
$n_{i\ell k}$	binary	cells with `encoding = "indicator"`	1 on a day without intervention, Eq. (16)
$\mu_{i\ell k}$	continuous, $[0, \tau_{i\ell}]$	every cell	tracked mean degradation
$v_{i\ell k}$	continuous, $[0, M_v]$	cells whose model or bound tracks a variance	tracked variance
$R_{i\ell k}$	continuous, $[0, M_R]$	rainflow cells with the Hoeffding bound	tracked Hoeffding budget
$K_{i\ell k}$	continuous, $[0, M_K]$	rainflow cells with the Chernoff bound	tracked Chernoff quantity

Variable	Shape, type	Built when	Meaning
$g^{\mu}_{i\ell k}$, $g^{v}_{i\ell k}$, $g^{R}_{i\ell k}$, $g^{K}_{i\ell k}$	continuous, range of the quantity	ARD_1 cells, one per tracked quantity except the Wiener variance	anchors, Eq. (11)
$w_{i\ell kp}$	binary, $p = 1, \dots, P$	cells with a quadratic row in the `pwl` form, $P \geq 2$	segment of the mean, Eq. (21)
$z_{i\ell k}$	continuous, ≥ 0	cells with $C_{R,i\ell} > 0$, on days whose cost enters the objective	degradation removed by a repair, Section 7.12
u	scalar, ≥ 0	$C_D > 0$	damage penalty, Section 7.13

M_v, M_R, M_K are the natural ranges of Section 7.4. The output reports the tracked values; each is an upper bound on the value that the rules give with every row at equality (scoping document, section "Stochastic loop constraint").

Objective Function

Cost of day k, and costs of the two phases:

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} C_{R,i\ell}\, z_{i\ell k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} C_{\mathrm{rep},i\ell}\, x^{r}_{i\ell k}, \tag{2}$$

$$J_{\mathrm{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\mathrm{op}} = \sum_{k=H_1+1}^{T} c_k. \tag{3}$$

$z_{i\ell k}$ reads 0 where it is not built; the last sum of c_k is absent when `allow_replacement = false`. The input `objective_mode` (default `total`) selects the objective:

$$\min \begin{cases} J_{\mathrm{init}} + J_{\mathrm{op}} + C_D\, u & \texttt{total}, \\ J_{\mathrm{op}}/H_2 + C_D\, u & \texttt{operating_average}, \\ J_{\mathrm{init}} + \dfrac{E - H_1}{H_2}\, J_{\mathrm{op}} + C_D\, u & \texttt{evaluation_total}, \end{cases} \tag{4}$$

with E the input `evaluation_horizon`, required in the mode `evaluation_total`, $E > H_1$. The term $C_D\, u$ is present only when $C_D > 0$.

Cost terms (shapes in Section 4).

- $C_M \geq 0$, scalar: paid once per train per depot day ($x_{i0k} = 1$), whatever the number of interventions on that day. An idle day costs nothing.
- $C_{R,i\ell} \geq 0$: per unit of mean degradation removed by a repair (Section 7.12).
- $C_{\mathrm{rep},i\ell} \geq 0$: per replacement; required, no default.
- $C_D \geq 0$: weight of u (Section 7.13); the term is added once in every mode.

Constraints

Every row holds for every day $k = 1, \dots, T$ unless its section says otherwise.

Assignment Constraints

Each train performs at most one activity per day:

$$\sum_{j=0}^{M} x_{ijk} \leq 1 \qquad \forall\, i, k. \tag{5}$$

Each mission is flown by exactly one train every day:

$$\sum_{i=1}^{F} x_{ijk} = 1 \qquad \forall\, j \in \{1, \dots, M\},\ k. \tag{6}$$

Exactly M trains fly every day; the other $F - M$ trains are in the depot or idle. No other fleet-availability row is built. An idle day is an all-zero assignment row. Depot days and idle days add no increment.

Depot-Day Compatibility

A component may be repaired or replaced only on a depot day of its train, and at most one of the two on the same day:

$$x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k} \qquad \forall\, i, \ell, k. \tag{7}$$

With `allow_replacement = false` the row reads $x^m_{i\ell k} \le x_{i0k}$. Several components of one train may receive interventions on the same depot day; a component without an intervention keeps its state.

Repair Laws

The input `repair_model` selects the law per cell (default `ardinf`); the repair efficiency is $\rho = \rho_{i\ell} \in (0, 1]$. With D^- the degradation before a repair on day k (end of day $k-1$) and D^+ the degradation after it:

$$\mathrm{ARD}_\infty \ (\texttt{ardinf}): \quad D^+ = (1-\rho)\, D^-, \tag{8}$$

$$\mathrm{ARD}_1 \ (\texttt{ard1}): \quad D^+ = D^- - \rho \left(D^- - D^{\mathrm{last}}\right), \tag{9}$$

with D^{last} the level just after the last intervention (repair or replacement); before the first intervention of the horizon, the level just after the last intervention before it. Every model accepts both laws.

Repair map. For a tracked quantity s with anchor g, a repair on day k gives

$$\psi(s_{k-1}, g_{k-1}) = \begin{cases} a\, s_{k-1} & \mathrm{ARD}_\infty, \\ a\, s_{k-1} + (1-a)\, g_{k-1} & \mathrm{ARD}_1, \end{cases} \tag{10}$$

with the repair factor $a = 1 - \rho$ for a mean and for K, $a = (1-\rho)^2$ for a variance and for R, and $a = 1$ for the Wiener variance (both laws). The model sections give the factor of each tracked quantity.

Anchors (ARD_1 only; one per tracked quantity, none for the Wiener variance):

$$g_0 = \text{starting anchor}, \qquad g_k = \begin{cases} s_k & \text{on a repair or replacement day,} \\ g_{k-1} & \text{otherwise.} \end{cases} \tag{11}$$

The starting anchors $g^\mu_{i\ell 0}$, $g^v_{i\ell 0}$, $g^R_{i\ell 0}$, $g^K_{i\ell 0}$ are the optional inputs `anchor_mu_0`, `anchor_v_0`, `anchor_R_0`, `anchor_K_0`; the default of each is the initial value of its quantity; each satisfies $0 \le g_0 \le s_0$ (Section 4.12).

Encoding of the Dynamics

Every dynamics row is one-sided ($\ge$); replacement rows and anchor rows are equalities. For one tracked quantity s of a cell (a mean, a variance, R or K): day increment δs_k, natural range M_s,

replacement value s^{new}, anchor g (ARD$_1$ only), repair map ψ (Eq. (10)), cell binaries x^m_k, x^r_k, and $e_k = x^m_k + x^r_k$. The input `encoding` selects the form per cell.

Big-M form (`encoding = "bigm"`, the default):

$$\text{no intervention:} \qquad s_k \geq s_{k-1} + \delta s_k - M_s\,(x^m_k + x^r_k), \tag{12}$$

$$\text{repair:} \qquad s_k \geq \psi(s_{k-1}, g_{k-1}) - M_s\,(1 - x^m_k), \tag{13}$$

$$\text{replacement:} \qquad |s_k - s^{\text{new}}| \leq M_s\,(1 - x^r_k), \tag{14}$$

$$\text{anchor (ARD}_1\text{):} \qquad |g_k - s_k| \leq M_s\,(1 - e_k), \qquad |g_k - g_{k-1}| \leq M_s\,e_k. \tag{15}$$

Each row with an absolute value is built as two linear rows.

Indicator form (`encoding = "indicator"`). One binary $n_{i\ell k}$ per cell and day, and Gurobi indicator constraints:

$$n_{i\ell k} + x^m_{i\ell k} + x^r_{i\ell k} = 1, \tag{16}$$

$$n_k = 1 \;\Rightarrow\; s_k \geq s_{k-1} + \delta s_k, \qquad x^m_k = 1 \;\Rightarrow\; s_k \geq \psi(s_{k-1}, g_{k-1}), \qquad x^r_k = 1 \;\Rightarrow\; s_k = s^{\text{new}}, \tag{17}$$

$$n_k = 1 \;\Rightarrow\; g_k = g_{k-1}, \qquad x^m_k = 1 \;\Rightarrow\; g_k = s_k, \qquad x^r_k = 1 \;\Rightarrow\; g_k = s_k. \tag{18}$$

Rules for both forms.

- Anchor rows only under ARD$_1$, for every tracked quantity except the Wiener variance.
- Wiener variance: $\psi = s_{k-1}$, no anchor; it is never left free below on a repair day. In the big-M form its no-intervention and repair rows are replaced by the single row Eq. (50).
- `allow_replacement = false`: no x^r; read $x^r_k = 0$ and drop the replacement rows.
- At binary points the two forms have the same feasible set.

Natural ranges M_s (big-M constants): $\tau_{i\ell}$ for a mean; $V_{\max}$ for a Gaussian or Wiener variance; V^{C} or V^{B} for a rainflow variance (Cantelli or Bernstein); $2\tau^2/\kappa$ for R; $s_{i\ell}\tau - \kappa$ for K (Section 7.11). An anchor has the range of its quantity.

Replacement

The input `allow_replacement` (default `true`) makes replacements available; when `false`, the binaries $x^r_{i\ell k}$, the replacement rows and the replacement cost are not built. On a replacement day every tracked quantity of the cell takes its replacement value, and every anchor the same value:

- $\mu^{\text{new}}_{i\ell}$: input `replacement_mu`, default 0;
- $v^{\text{new}}_{i\ell}$: input `replacement_v`, default 0, where a variance is tracked;
- $R^{\text{new}}_{i\ell}$, $K^{\text{new}}_{i\ell}$: rainflow Hoeffding and Chernoff, Section 7.11.

The rows are Eq. (14) or Eq. (17). Each replacement value lies in the natural range of its quantity, and the replacement state satisfies every row of a replacement day in the form built at the cell (Section 4.12).

The input `replacement_as_new` (default `false`) sets every replacement value of the cell to 0; a non-zero replacement input together with it is an input error. A replacement costs $C_{\text{rep},i\ell}$ only ($z_{i\ell k} = 0$ on a replacement day). A cell may be replaced on any depot day, any number of times.

Linear Forms of the Reliability Rows

The Gaussian, Wiener, Cantelli, Hoeffding and Bernstein rows have the form

$$\chi_{i\ell k} \;\leq\; f(\mu_{i\ell k}), \qquad f(\mu) = c_2\,(\tau - \mu)^2 + c_1\,(\tau - \mu), \qquad \mu_{i\ell k} \leq \bar{\mu}_{\text{side}}, \tag{19}$$

with $f'(\mu) = -2c_2(\tau - \mu) - c_1$; the third relation is the side row.

Row	χ	c_2	c_1	$\bar{\mu}_{\text{side}}$
Gaussian, Wiener	v	$1/\kappa_G^2$	0	τ
Rainflow, Cantelli	v	$1/\kappa_C^2 = \varepsilon/(1-\varepsilon)$	0	τ
Rainflow, Hoeffding	R	$2/\kappa$	0	τ
Rainflow, Bernstein	v	$1/(2\kappa)$	$-b_{i\ell}/3$	$\tau - c_{i\ell}$

The input `reliability_impl` selects the form per cell; every form keeps the side row.

- `exact`: Eq. (19), a nonconvex quadratic constraint. Gurobi's `NonConvex` is set to 2 when at least one cell has a quadratic row in this form.
- `tangent`: one linear row per day, at $\mu_r = \texttt{tangent_ref}_{i\ell} \cdot \tau_{i\ell}$ (`tangent_ref` per cell, in $[0, 1)$, default 0.5):

$$\chi_{i\ell k} \;\le\; f(\mu_r) + f'(\mu_r)\,(\mu_{i\ell k} - \mu_r). \tag{20}$$

  At a Bernstein cell, $\mu_r < \sqrt{\tau\,(\tau - 2c_{i\ell})}$ (Section 4.12).
- `pwl`: $P = \texttt{pwl_points}$ equal segments $[(p-1)\tau/P,\ p\tau/P]$ of $[0, \tau]$ (default $P = 8$), midpoints $\hat{\mu}_p = (p - \frac{1}{2})\tau/P$, and one binary $w_{i\ell kp}$ per cell, day and segment:

$$\textstyle\sum_{p=1}^{P} w_{i\ell kp} = 1, \tag{21}$$

$$w_{i\ell kp} = 1 \;\Rightarrow\; (p-1)\,\tau/P \le \mu_{i\ell k} \le p\,\tau/P, \tag{22}$$

$$w_{i\ell kp} = 1 \;\Rightarrow\; \chi_{i\ell k} \le f(\hat{\mu}_p) + f'(\hat{\mu}_p)\,(\mu_{i\ell k} - \hat{\mu}_p). \tag{23}$$

  The implications are Gurobi indicator constraints in both encodings. With $P = 1$: the single tangent at $\tau/2$, no binaries. At a Bernstein cell, $\tau/(2P) < \sqrt{\tau\,(\tau - 2c_{i\ell})}$ (Section 4.12).

At a cell whose reliability row is linear (Markov, Chernoff, inverse Gaussian, Gamma) the row is built in its exact form and the value of `reliability_impl` is ignored. An unknown value is an input error at every cell. Each linear form implies the exact row (scoping document, section "Linear forms of the reliability rows").

Gaussian Model

For cells (i, ℓ) with `model` = `gaussian`. In this section and the next four, the subscript $i\ell$ is dropped and k is the day.

Model. A mission increment is $\mathcal{N}(\mu_{jk}^{\text{inc}}, v_{jk}^{\text{inc}})$ (inputs `mu`, `v`); the initial damage is $\mathcal{N}(\mu_0, v_0)$; a replacement draw is $\mathcal{N}(\mu^{\text{new}}, v^{\text{new}})$. A known damage has variance 0. **Assumption N1.** The increments, the initial damage and the replacement draws are mutually independent.

Laws and tracked quantities. ARD_∞ and ARD_1. Tracked: μ_k and v_k; under ARD_1 also the anchors g_k^μ and g_k^v. Day maps:

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, } \text{ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^{\mu} & \text{repair, } \text{ARD}_1, \\ \mu^{\text{new}} & \text{replacement}, \end{cases} \tag{24}$$

$$v_k = \begin{cases} v_{k-1} + \delta v_k & \text{no intervention on day } k, \\ (1-\rho)^2\, v_{k-1} & \text{repair, } \text{ARD}_\infty, \\ (1-\rho)^2\, v_{k-1} + \left(1 - (1-\rho)^2\right) g_{k-1}^{v} & \text{repair, } \text{ARD}_1, \\ v^{\text{new}} & \text{replacement}, \end{cases} \tag{25}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu_{jk}^{\text{inc}}$ and $\delta v_k = \sum_{j=1}^{M} x_{ijk}\,v_{jk}^{\text{inc}}$.

Big-M rows (`encoding = "bigm"`, the default). In this section and the next four, each row holds for $k = 1, \ldots, T$, and $e_k = x_k^m + x_k^r$. The ARD_∞ rows are built at ARD_∞ cells; the ARD_1 rows and the anchor rows at ARD_1 cells. With `allow_replacement = false`, $x_k^r = 0$ and the replacement rows are not built. Big-M constants: τ for μ and g^μ; $V_{\max} = \tau^2/\kappa_G^2$ for v and g^v, with $\kappa_G = \Phi^{-1}(1-\varepsilon)$.

Mean.

$$\begin{aligned}
&\text{no intervention:} && \mu_k \geq \mu_{k-1} + \delta\mu_k - \tau\,(x_k^m + x_k^r), && (26)\\
&\text{repair, } \text{ARD}_\infty\text{:} && \mu_k \geq (1-\rho)\,\mu_{k-1} - \tau\,(1 - x_k^m), && (27)\\
&\text{repair, } \text{ARD}_1\text{:} && \mu_k \geq (1-\rho)\,\mu_{k-1} + \rho\,g_{k-1}^{\mu} - \tau\,(1 - x_k^m), && (28)\\
&\text{replacement:} && \mu_k \geq \mu^{\text{new}} - \tau\,(1 - x_k^r), \quad \mu_k \leq \mu^{\text{new}} + \tau\,(1 - x_k^r). && (29)
\end{aligned}$$

Variance.

$$\begin{aligned}
&\text{no intervention:} && v_k \geq v_{k-1} + \delta v_k - V_{\max}\,(x_k^m + x_k^r), && (30)\\
&\text{repair, } \text{ARD}_\infty\text{:} && v_k \geq (1-\rho)^2\,v_{k-1} - V_{\max}\,(1 - x_k^m), && (31)\\
&\text{repair, } \text{ARD}_1\text{:} && v_k \geq (1-\rho)^2\,v_{k-1} + \left(1 - (1-\rho)^2\right) g_{k-1}^{v} - V_{\max}\,(1 - x_k^m), && (32)\\
&\text{replacement:} && v_k \geq v^{\text{new}} - V_{\max}\,(1 - x_k^r), \quad v_k \leq v^{\text{new}} + V_{\max}\,(1 - x_k^r). && (33)
\end{aligned}$$

Anchors (ARD_1), for $(y, M_y) \in \{(\mu, \tau),\ (v, V_{\max})\}$:

$$\begin{aligned}
g_k^y - y_k &\leq M_y\,(1 - e_k), & y_k - g_k^y &\leq M_y\,(1 - e_k), && (34)\\
g_k^y - g_{k-1}^y &\leq M_y\,e_k, & g_{k-1}^y - g_k^y &\leq M_y\,e_k. && (35)
\end{aligned}$$

Day 0 and replacement. The constants μ_0, v_0 and, under ARD_1, g_0^μ, g_0^v (Section 7.3); the replacement values μ^{new}, v^{new} (Section 7.5).

Indicator form (`encoding = "indicator"`): the rows Eqs. (16)–(18) with the same day maps, Eqs. (24)–(25).

Reliability row

Requires $\varepsilon < 0.5$ (Section 4.12). On every day $k = 1, \ldots, T$:

$$\kappa_G^2\,v_k \leq (\tau - \mu_k)^2, \qquad \mu_k \leq \tau. \tag{36}$$

Forms. `exact`: Eq. (36) as written, a nonconvex quadratic constraint (`NonConvex` = 2, Section 7.6). `tangent` and `pwl`: Section 7.6, with $\chi = v$, $f(\mu) = (\tau - \mu)^2/\kappa_G^2$ and the side row $\mu \leq \tau$.

Optional row. If `v_max_user` is given for the cell: $v_k \leq V^{\text{user}}$ on every day.

Inverse Gaussian Model

For cells (i, ℓ) with `model` = `inverse_gaussian` (IG).

Model. $\text{IG}(\mu, \lambda)$ is the inverse Gaussian law with mean μ and shape λ. Every term of mean a (a mission increment, mean μ_{jk}^{inc}; the initial damage, μ_0; a replacement draw, μ^{new}) has the law $\text{IG}(a, a/\eta) = a\,Z$, with $Z \sim \text{IG}(1, 1/\eta)$ and $\eta > 0$ the dispersion of the cell (input `eta`). A

known damage is a constant. No independence between terms is required (scoping document, section "Stochastic orderings").

Laws and tracked quantities. ARD_∞ and ARD_1. Tracked: μ_k; under ARD_1 also the anchor g^μ_k. Day map:

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, } \mathrm{ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g^\mu_{k-1} & \text{repair, } \mathrm{ARD}_1, \\ \mu^{\mathrm{new}} & \text{replacement}, \end{cases} \tag{37}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu^{\mathrm{inc}}_{jk}$.

Big-M rows (`encoding = "bigm"`, the default; conditions in Section 7.7). Big-M constant: τ for μ and g^μ.

$$\text{no intervention:} \quad \mu_k \geq \mu_{k-1} + \delta\mu_k - \tau\,(x^m_k + x^r_k), \tag{38}$$

$$\text{repair, } \mathrm{ARD}_\infty\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} - \tau\,(1 - x^m_k), \tag{39}$$

$$\text{repair, } \mathrm{ARD}_1\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} + \rho\, g^\mu_{k-1} - \tau\,(1 - x^m_k), \tag{40}$$

$$\text{replacement:} \quad \mu_k \geq \mu^{\mathrm{new}} - \tau\,(1 - x^r_k), \quad \mu_k \leq \mu^{\mathrm{new}} + \tau\,(1 - x^r_k). \tag{41}$$

Anchor (ARD_1):

$$g^\mu_k - \mu_k \leq \tau\,(1 - e_k), \qquad \mu_k - g^\mu_k \leq \tau\,(1 - e_k), \tag{42}$$

$$g^\mu_k - g^\mu_{k-1} \leq \tau\, e_k, \qquad g^\mu_{k-1} - g^\mu_k \leq \tau\, e_k. \tag{43}$$

Day 0 and replacement. The constants μ_0 and, under ARD_1, g^μ_0 (Section 7.3); the replacement value μ^{new} (Section 7.5).

Indicator form (`encoding = "indicator"`): the rows Eqs. (16)–(18) with the same day map, Eq. (37).

Reliability row

Let q be the $(1-\varepsilon)$-quantile of Z, $\Pr(Z > q) = \varepsilon$, and $C = \mathbb{E}[Z \mid Z > q]$. On every day $k = 1, \ldots, T$:

$$\mu_k \leq \bar{\mu}, \qquad \bar{\mu} = \frac{\tau}{C}. \tag{44}$$

The row is linear: it is built in its exact form, and a value of `reliability_impl` at the cell is ignored.

Computation of $\bar{\mu}$ (offline, once per cell). With $r(x) = 1/\sqrt{\eta x}$, the distribution function of Z is

$$F(x) = \Phi\big(r(x)\,(x-1)\big) + e^{2/\eta}\,\Phi\big(-r(x)\,(x+1)\big).$$

The second term is computed as $\exp\big(2/\eta + \ln\Phi(-r(x)(x+1))\big)$, with $\ln\Phi$ from a log-space routine (`scipy.special.log_ndtr`).

1. q: the root of $1 - F(q) = \varepsilon$, by bisection or `brentq` on $\ln q$; equivalently, the method `isf` of `scipy.stats.invgauss` with `mu` $= \eta$, `scale` $= 1/\eta$.
2. The tail mean: $\mathbb{E}[Z;\, Z > q] = F(1/q)$, so $\bar{\mu} = \tau\,\varepsilon / F(1/q)$. Quadrature of $\int_q^\infty z\, f_Z(z)\, dz$, with $f_Z(z) = (2\pi\eta z^3)^{-1/2} \exp\big(-(z-1)^2/(2\eta z)\big)$, gives the same value.

Wiener Model

For cells (i, ℓ) with `model = wiener`.

Model. A mission day adds $\mathcal{N}(\mu_{jk}^{\text{inc}}, \sigma^2)$, with the mean from `mu` and one diffusion constant $\sigma > 0$ per cell (input `sigma`); the initial damage is $\mathcal{N}(\mu_0, v_0)$, $v_0 \geq 0$; a replacement draw is $\mathcal{N}(\mu^{\text{new}}, v^{\text{new}})$. **Assumption W1.** These terms are mutually independent. A repair acts on the mean only; the variance falls only at a replacement.

Laws and tracked quantities. ARD_∞ and ARD_1. Tracked: μ_k and v_k; under ARD_1 also the mean anchor g_k^μ (no variance anchor). The mean follows Eq. (24). The variance follows

$$v_k = \begin{cases} v_{k-1} + \delta v_k & \text{no intervention on day } k, \\ v_{k-1} & \text{repair, both laws,} \\ v^{\text{new}} & \text{replacement,} \end{cases} \qquad \delta v_k = \sigma^2 \sum_{j=1}^{M} x_{ijk}. \tag{45}$$

Big-M rows (`encoding = "bigm"`, the default; conditions in Section 7.7). Big-M constants: τ for μ and g^μ; $V_{\max} = \tau^2/\kappa_G^2$ for v, with $\kappa_G = \Phi^{-1}(1-\varepsilon)$.

Mean.

$$\text{no intervention:} \quad \mu_k \geq \mu_{k-1} + \delta\mu_k - \tau\,(x_k^m + x_k^r), \tag{46}$$

$$\text{repair, } \text{ARD}_\infty\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} - \tau\,(1 - x_k^m), \tag{47}$$

$$\text{repair, } \text{ARD}_1\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^\mu - \tau\,(1 - x_k^m), \tag{48}$$

$$\text{replacement:} \quad \mu_k \geq \mu^{\text{new}} - \tau\,(1 - x_k^r), \quad \mu_k \leq \mu^{\text{new}} + \tau\,(1 - x_k^r). \tag{49}$$

Variance (both laws; no anchor).

$$\text{no intervention, repair:} \quad v_k \geq v_{k-1} + \delta v_k - V_{\max}\, x_k^r, \tag{50}$$

$$\text{replacement:} \quad v_k \geq v^{\text{new}} - V_{\max}\,(1 - x_k^r), \quad v_k \leq v^{\text{new}} + V_{\max}\,(1 - x_k^r). \tag{51}$$

Anchor (ARD_1, mean only):

$$g_k^\mu - \mu_k \leq \tau\,(1 - e_k), \qquad \mu_k - g_k^\mu \leq \tau\,(1 - e_k), \tag{52}$$

$$g_k^\mu - g_{k-1}^\mu \leq \tau\, e_k, \qquad g_{k-1}^\mu - g_k^\mu \leq \tau\, e_k. \tag{53}$$

Day 0 and replacement. The constants μ_0, v_0 and, under ARD_1, g_0^μ (Section 7.3); the replacement values μ^{new}, v^{new} (Section 7.5).

Indicator form (`encoding = "indicator"`): the rows Eqs. (16)–(18) with the same day maps, Eqs. (24) and (45): the variance has $\psi = v_{k-1}$, and only g^μ has anchor rows.

Reliability row. The Gaussian row, Eq. (36), with $\kappa_G = \Phi^{-1}(1-\varepsilon)$, $\varepsilon < 0.5$, $V_{\max} = \tau^2/\kappa_G^2$, on every day $k = 1, \ldots, T$, in the same forms and with the same f and side row (Section 7.7). The optional row $v_k \leq V^{\text{user}}$ is added when `v_max_user` is given.

Note. The loop row $v_T \leq v_{H_1}$ forces a replacement of the component in the operating phase whenever its train flies a mission there; with `allow_replacement` false, the train cannot fly in the operating phase (scoping document, section "Stochastic orderings", loop constraint for each degradation model).

Gamma Model

For cells (i, ℓ) with `model` = `gamma`.

Model. Gamma(a, β) has shape a, rate β and mean a/β. Terms:

- a mission increment with mean $\mu_{jk}^{\text{inc}} > 0$ (`mu`) and rate β_{jk} (`gamma_beta`, `gamma_beta_trans`): Gamma$(\beta_{jk}\mu_{jk}^{\text{inc}}, \beta_{jk})$;
- the initial damage, $\mu_0 > 0$: Gamma$(\beta_0\mu_0, \beta_0)$ when `gamma_beta_0` $= \beta_0$ is given; the known constant μ_0 when it is omitted;
- a replacement draw, $\mu^{\text{new}} > 0$: Gamma$(\beta^{\text{new}}\mu^{\text{new}}, \beta^{\text{new}})$ when `gamma_beta_new` $= \beta^{\text{new}}$ is given; the known constant μ^{new} when it is omitted.

Assumption G1. All these terms are mutually independent.

Laws and tracked quantities. ARD$_\infty$ and ARD$_1$. Tracked: μ_k (the shape is not tracked); under ARD$_1$ also the anchor g_k^μ. Day map:

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^\mu & \text{repair, ARD}_1, \\ \mu^{\text{new}} & \text{replacement}, \end{cases} \tag{54}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu_{jk}^{\text{inc}}$.

Big-M rows (`encoding = "bigm"`, the default; conditions in Section 7.7). Big-M constant: τ for μ and g^μ.

$$\text{no intervention:} \quad \mu_k \geq \mu_{k-1} + \delta\mu_k - \tau\,(x_k^m + x_k^r), \tag{55}$$

$$\text{repair, ARD}_\infty\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} - \tau\,(1 - x_k^m), \tag{56}$$

$$\text{repair, ARD}_1\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^\mu - \tau\,(1 - x_k^m), \tag{57}$$

$$\text{replacement:} \quad \mu_k \geq \mu^{\text{new}} - \tau\,(1 - x_k^r), \quad \mu_k \leq \mu^{\text{new}} + \tau\,(1 - x_k^r). \tag{58}$$

Anchor (ARD$_1$):

$$g_k^\mu - \mu_k \leq \tau\,(1 - e_k), \qquad \mu_k - g_k^\mu \leq \tau\,(1 - e_k), \tag{59}$$

$$g_k^\mu - g_{k-1}^\mu \leq \tau\, e_k, \qquad g_{k-1}^\mu - g_k^\mu \leq \tau\, e_k. \tag{60}$$

Day 0 and replacement. The constants μ_0 and, under ARD$_1$, g_0^μ (Section 7.3); the replacement value μ^{new} (Section 7.5).

Indicator form (`encoding = "indicator"`): the rows Eqs. (16)–(18) with the same day map, Eq. (54).

Smallest rate.

$$\beta^* = \min\Big(\{\beta_{jk} : \mu_{jk}^{\text{inc}} > 0\} \cup \{\beta_0 : \mu_0 > 0\} \cup \{\beta^{\text{new}} : \mu^{\text{new}} > 0\}\Big), \tag{61}$$

over the missions j and the days k of both phases. β_0 and β^{new} enter only when given, and β^{new} only when `allow_replacement` is true. A cell in which no term has a rate has the row $\mu_k \leq \tau$ in place of Eq. (62).

Reliability row

$Q(a, x) = \Pr(\text{Gamma}(a, 1) > x)$ is the regularised upper incomplete gamma function (`gammaincc` of `scipy.special`). Let $t = \beta^* \tau$ and, for $A \in [0, t)$,

$$w(A) = \max_{c \in (0,1]} Q\left(\frac{A}{c}, \frac{t}{c}\right).$$

The sharp cap $A_1(t, \varepsilon)$ is the largest $A \in [0, t)$ with $w(A) \le \varepsilon$. For $\varepsilon \ge 1/2$ every $A < t$ satisfies this, and A_1 is the value that step 3 below returns, which lies below t. On every day $k = 1, \ldots, T$:

$$\mu_k \le \frac{A_1(\beta^* \tau,\, \varepsilon)}{\beta^*}. \tag{62}$$

The row is linear: it is built in its exact form, and a value of `reliability_impl` at the cell is ignored.

Computation of A_1 (offline, once per cell):

1. $\hat{A}$: the root of $Q(\hat{A}, t) = \varepsilon$, by bisection on $\hat{A}$ (`gammaincinv` solves for the second argument and does not apply).
2. $w(A)$: maximum of $c \mapsto Q(A/c, t/c)$ over a logarithmic grid of at least 400 values of c in $[\min(10^{-4}, t/100), 1]$, refined by a golden-section search between the neighbours of the best grid point. If the best grid point is the smallest c, the grid is extended downwards.
3. If $\hat{A} < t$ and $w(\hat{A}) \le \varepsilon$: $A_1 = \hat{A}$. Otherwise: bisection on A in $[\varepsilon t,\, \min(\hat{A}, t))$, to a relative tolerance of 10^{-9}; $w(\varepsilon t) \le \varepsilon$ holds. The bisection returns the lower end of its last interval.

Rainflow Model

For cells (i, ℓ) with `model` = `rainflow`. The law of a mission increment comes from rainflow counting of recorded load histories (Palmgren–Miner damage) and enters only through summary statistics: mean, variance, width of its range, or cumulant generating function (CGF) at a fixed s. The tail bound is chosen per cell by `bound_method`: `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`.

Assumptions on the input data.

- **A0.** Every increment, the initial damage and every replacement draw are non-negative.
- **A1.** The mission increments, the initial damage and the replacement draws are mutually independent (all bounds except Markov).
- **A2** (Hoeffding, Bernstein). Each increment lies in an interval of width b_{jk}^{inc} (input `support`). The initial damage is a known part plus independent terms in intervals, with sum of squared widths R_0 (`R_0`) and largest width b^0 (`support_0`); a replacement draw likewise, with R^{new}, b^{new} (`R_new`, `support_new`). A known damage has $R = b = 0$, the defaults.
- **A3.** After day T the operating phase repeats with period H_2 (Section 7.14).

The *width* of X is the width b of an interval $[l, l + b]$ that contains X almost surely; it must satisfy $v \le b^2/4$ (Section 4.12).

Tracked quantities per bound. Every cell tracks μ; the bound adds at most one quantity; under ARD_1 each tracked quantity has an anchor (g^μ and g^v, g^R or g^K). Both laws are accepted for every bound. "Forms" lists the forms in which the row is built ("all three": `exact`, `tangent`, `pwl`); at a linear row a value of `reliability_impl` is ignored.

`bound_method`	Tracked	Increment inputs	Initial; replacement inputs	Forms
`markov`	μ	`mu`	–	`exact`

`cantelli`	μ, v	`mu`, `v`	`v_0`; `replacement_v`	all three
`hoeffding`	μ, R	`mu`, `support`	`R_0`; `R_new`	all three
`bernstein`	μ, v	`mu`, `v`, `support`	`v_0`, `support_0`; `replacement_v`, `support_new`	all three
`chernoff`	μ, K	`mu`, `cgf`, `s_chernoff`	`cgf_0`; `cgf_new`	`exact`

R is the Hoeffding budget (sum of the squared widths of the terms of the damage). K is the CGF of the damage at $s =$ `s_chernoff` > 0. Day increments: $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu_{jk}^{\text{inc}}$, $\delta v_k = \sum_j x_{ijk}\,v_{jk}^{\text{inc}}$, $\delta R_k = \sum_j x_{ijk}\,(b_{jk}^{\text{inc}})^2$, $\delta K_k = \sum_j x_{ijk}\,K_{jk}^{\text{inc}}$, with $K_{jk}^{\text{inc}} = \ln \mathbb{E}\, e^{sX_{jk}}$ (input `cgf`).

Day maps. Each tracked quantity follows the generic map of Section 7.4 with its factor a: $1-\rho$ for μ and K, $(1-\rho)^2$ for v and R. It adds its day increment on a day without intervention; a repair multiplies it by a under ARD_∞, and gives a times its value plus $1-a$ times its anchor under ARD_1; a replacement sets it to its replacement value. Its anchor is its value just after the last intervention.

Starting and replacement values. μ and v: from the inputs (Section 7.5). R_0 from `R_0` and R^{new} from `R_new`, default 0. K_0 from `cgf_0`, default $s\,\mu_0$; K^{new} from `cgf_new`, default $s\,\mu^{\text{new}}$. With `replacement_as_new`, all replacement values are 0. The defaults describe a known damage; a random initial or replacement damage needs its own `R_0`, `cgf_0`, `R_new`, `cgf_new` (scoping document, section "Conservative surrogates for the empirical damage-increment distribution").

Big-M rows (`encoding = "bigm"`, the default; conditions in Section 7.7), with $\kappa = \ln(1/\varepsilon)$ and c from Eq. (84). The rows of μ are built at every cell, the rows of v at Cantelli and Bernstein cells, the rows of R at Hoeffding cells, and the rows of K at Chernoff cells. Big-M constants: τ for μ and g^μ; $M_v = V^{\text{C}} = \varepsilon\tau^2/(1-\varepsilon)$ (Cantelli) or $M_v = V^{\text{B}} = \tau(\tau - 2c)/(2\kappa)$ (Bernstein) for v and g^v; $2\tau^2/\kappa$ for R and g^R; $s\tau - \kappa$ for K and g^K.

Mean (every bound).

$$\text{no intervention:} \quad \mu_k \geq \mu_{k-1} + \delta\mu_k - \tau\,(x_k^m + x_k^r), \tag{63}$$

$$\text{repair, } \text{ARD}_\infty\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} - \tau\,(1 - x_k^m), \tag{64}$$

$$\text{repair, } \text{ARD}_1\text{:} \quad \mu_k \geq (1-\rho)\,\mu_{k-1} + \rho\,g_{k-1}^{\mu} - \tau\,(1 - x_k^m), \tag{65}$$

$$\text{replacement:} \quad \mu_k \geq \mu^{\text{new}} - \tau\,(1 - x_k^r), \quad \mu_k \leq \mu^{\text{new}} + \tau\,(1 - x_k^r). \tag{66}$$

Variance (Cantelli, Bernstein).

$$\text{no intervention:} \quad v_k \geq v_{k-1} + \delta v_k - M_v\,(x_k^m + x_k^r), \tag{67}$$

$$\text{repair, } \text{ARD}_\infty\text{:} \quad v_k \geq (1-\rho)^2\,v_{k-1} - M_v\,(1 - x_k^m), \tag{68}$$

$$\text{repair, } \text{ARD}_1\text{:} \quad v_k \geq (1-\rho)^2\,v_{k-1} + \left(1 - (1-\rho)^2\right) g_{k-1}^{v} - M_v\,(1 - x_k^m), \tag{69}$$

$$\text{replacement:} \quad v_k \geq v^{\text{new}} - M_v\,(1 - x_k^r), \quad v_k \leq v^{\text{new}} + M_v\,(1 - x_k^r). \tag{70}$$

Hoeffding budget (Hoeffding).

$$\text{no intervention:} \quad R_k \geq R_{k-1} + \delta R_k - (2\tau^2/\kappa)\,(x_k^m + x_k^r), \tag{71}$$

$$\text{repair, } \text{ARD}_\infty\text{:} \quad R_k \geq (1-\rho)^2\,R_{k-1} - (2\tau^2/\kappa)\,(1 - x_k^m), \tag{72}$$

$$\text{repair, } \text{ARD}_1\text{:} \quad R_k \geq (1-\rho)^2\,R_{k-1} + \left(1 - (1-\rho)^2\right) g_{k-1}^{R} - (2\tau^2/\kappa)\,(1 - x_k^m), \tag{73}$$

$$\text{replacement:} \quad R_k \geq R^{\text{new}} - (2\tau^2/\kappa)\,(1 - x_k^r), \quad R_k \leq R^{\text{new}} + (2\tau^2/\kappa)\,(1 - x_k^r). \tag{74}$$

Chernoff quantity (Chernoff).

$$\text{no intervention:} \quad K_k \geq K_{k-1} + \delta K_k - (s\tau - \kappa)\,(x_k^m + x_k^r), \tag{75}$$
$$\text{repair, ARD}_\infty\text{:} \quad K_k \geq (1-\rho)\,K_{k-1} - (s\tau - \kappa)\,(1 - x_k^m), \tag{76}$$
$$\text{repair, ARD}_1\text{:} \quad K_k \geq (1-\rho)\,K_{k-1} + \rho\, g_{k-1}^K - (s\tau - \kappa)\,(1 - x_k^m), \tag{77}$$
$$\text{replacement:} \quad K_k \geq K^{\text{new}} - (s\tau - \kappa)\,(1 - x_k^r), \quad K_k \leq K^{\text{new}} + (s\tau - \kappa)\,(1 - x_k^r). \tag{78}$$

Anchors (ARD$_1$), for each pair $(y, M_y) \in \{(\mu, \tau),\ (v, M_v),\ (R, 2\tau^2/\kappa),\ (K, s\tau - \kappa)\}$ whose quantity y the cell tracks:

$$g_k^y - y_k \leq M_y\,(1 - e_k), \qquad y_k - g_k^y \leq M_y\,(1 - e_k), \tag{79}$$
$$g_k^y - g_{k-1}^y \leq M_y\,e_k, \qquad g_{k-1}^y - g_k^y \leq M_y\,e_k. \tag{80}$$

Day 0 and replacement. For the quantities that the cell tracks: the constants μ_0, v_0, R_0, K_0 and, under ARD$_1$, g_0^μ, g_0^v, g_0^R, g_0^K (Section 7.3); the replacement values μ^{new}, v^{new}, R^{new}, K^{new} (paragraph "Starting and replacement values").

Indicator form (`encoding = "indicator"`): the rows Eqs. (16)–(18) with the same day maps (paragraph "Day maps").

Reliability rows

Each row holds on every day $k = 1, \ldots, T$.

Markov (`markov`):

$$\mu_k \leq \varepsilon\,\tau. \tag{81}$$

Cantelli (`cantelli`), with $\kappa_C = \sqrt{(1-\varepsilon)/\varepsilon}$:

$$\kappa_C^2\, v_k \leq (\tau - \mu_k)^2, \qquad \mu_k \leq \tau. \tag{82}$$

Hoeffding (`hoeffding`):

$$(\tau - \mu_k)^2 \geq \frac{\kappa}{2}\, R_k, \qquad \mu_k \leq \tau. \tag{83}$$

Bernstein (`bernstein`). Support constant, precomputed per cell:

$$b = \max\Big(b^0,\ b^{\text{new}},\ \max_{j,k} b_{jk}^{\text{inc}}\Big), \qquad c = \frac{\kappa\, b}{3}, \tag{84}$$

over the missions j and the days k of both phases; b^{new} enters only when `allow_replacement` is true. Row, with its side row:

$$(\tau - c - \mu_k)^2 \geq c^2 + 2\kappa\, v_k, \qquad \mu_k \leq \tau - c. \tag{85}$$

Requires $\tau > 2\kappa b/3$ (Section 4.12).

Chernoff (`chernoff`), with $s =$ `s_chernoff`:

$$K_k - s\,\tau \leq \ln \varepsilon. \tag{86}$$

Requires $s > \kappa/\tau$ and $K^{\text{new}} \leq s\tau - \kappa$ (Section 4.12).

Forms. Markov and Chernoff rows are linear and built in their exact form. The Cantelli, Hoeffding and Bernstein rows take the forms of Section 7.6 with:

Bound	χ	$f(\mu)$	Side row	$f(0)$
Cantelli	v	$\varepsilon(\tau-\mu)^2/(1-\varepsilon)$	$\mu \le \tau$	$\varepsilon\tau^2/(1-\varepsilon)$
Hoeffding	R	$2(\tau-\mu)^2/\kappa$	$\mu \le \tau$	$2\tau^2/\kappa$
Bernstein	v	$\big((\tau-\mu)^2 - 2c(\tau-\mu)\big)/(2\kappa)$	$\mu \le \tau - c$	$\tau(\tau-2c)/(2\kappa)$

At a Bernstein cell with $b > 0$ the tangent point must satisfy $\mu_r < \sqrt{\tau(\tau-2c)}$, and the `pwl` form requires $\tau/(2P) < \sqrt{\tau(\tau-2c)}$, $P =$ `pwl_points` (Section 4.12).

Optional row. If `v_max_user` is given for a Cantelli or Bernstein cell: $v_k \le V^{\text{user}}$ on every day.

Repair Cost Variable

$z_{i\ell k} \ge 0$ is the mean degradation removed by a repair of cell (i,ℓ) on day k, for every model and both laws. It is built at cells with $C_{R,i\ell} > 0$, on every day $k = 1,\dots,T$ except the transitory days in the mode `operating_average`. Only the lower rows are built:

$$z_{i\ell k} \;\ge\; \rho_{i\ell}\left(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1}\right) - \rho_{i\ell}\,\tau_{i\ell}\left(1 - x^{m}_{i\ell k}\right), \qquad z_{i\ell k} \ge 0, \tag{87}$$

with $g^{\mu} \equiv 0$ under ARD_∞. At the optimum $z_{i\ell k} = 0$ on every day without a repair, replacement days included. z is an upper bound on the expected removed degradation, except at the first repair when the starting anchor is above the true one (scoping document, section "Cost function").

Damage Penalty

The scalar $u \ge 0$ and its rows are built only when $C_D > 0$; with $C_D = 0$ the input `penalty_type` is ignored. One row per train and day:

$$\texttt{inf_norm}\ \text{(default)}: \quad u \;\ge\; \sum_{\ell=1}^{L} \mu_{i\ell k} \qquad \forall\, i,k, \tag{88}$$

$$\texttt{quadratic}: \quad u \;\ge\; \sum_{\ell=1}^{L} \mu^2_{i\ell k} \qquad \forall\, i,k. \tag{89}$$

At the optimum $u = \max_{i,k} \sum_\ell \mu_{i\ell k}$ (`inf_norm`) or $u = \max_{i,k} \sum_\ell \mu^2_{i\ell k}$ (`quadratic`): the largest damage of one train on one day. Eq. (89) is a rotated second-order cone; it does not need `NonConvex`. If $C_D > 0$ and `penalty_type = "quadratic"`, every cell with a quadratic reliability row must have `reliability_impl = "exact"`; otherwise a `ValueError` is raised (Section 4.12). At a cell with a linear row the value is ignored. With $C_D = 0$ the rule does not apply.

Loop Constraint

One hard row per cell and per coordinate s of its tracked state:

$$s_{i\ell T} \;\le\; s_{i\ell H_1}. \tag{90}$$

The rows cover every tracked quantity of the cell (the mean, and the variance, R or K where tracked) and, under ARD_1, every anchor. Each coordinate has its own row; no row compares a scalar function of them, such as a failure-probability bound. There is no soft loop penalty. With these rows every day of every repetition of the operating phase satisfies the reliability requirement, for every model, both laws, both encodings, every form, and any H_1, H_2 (scoping document, section "Stochastic loop constraint").

Consequences. An ARD_1 cell with $\rho_{i\ell} < 1$ whose train flies, in the operating phase, a mission with a positive mean increment for the cell must be replaced at least once in that phase; with `allow_`

`replacement = false` its train cannot fly such a mission there, and the model is infeasible if the fleet cannot cover the missions otherwise (scoping document, section "Stochastic loop constraint"). A Wiener cell likewise needs a replacement in the operating phase when its train flies a mission there (Section 7.9).

Full Optimization Problem

This section collects the whole problem in one place. Each row is stated once, in the section that the reference names; this section does not repeat the rows.

Objective. Minimise Eq. (4), the mode chosen by `objective_mode`, with the day cost Eq. (2).

Fleet rows. For every train i and day k: one activity, Eq. (5). For every mission $j \geq 1$ and day k: coverage, Eq. (6). For every cell and day: depot-day compatibility, Eq. (7).

Rows of a cell. For every cell (i, ℓ), with the model, repair law, tail bound, encoding and reliability form of the cell:

1. for every tracked quantity of the table below: the big-M rows of its model section (Sections 7.7–7.11; `encoding = "bigm"`) or the indicator rows Eqs. (16)–(17) (`encoding = "indicator"`);
2. under ARD_1, for every tracked quantity except the Wiener variance: the anchor rows of its model section, or Eq. (18);
3. the reliability row of the table, on every day $k = 1, \ldots, T$, in the form `exact`, `tangent` (Eq. (20)) or `pwl` (Eqs. (21)–(23)) for a quadratic row, and in its exact form for a linear row;
4. the loop rows Eq. (90), one per tracked quantity and per anchor;
5. where $C_{R,i\ell} > 0$: the repair-cost rows Eq. (87).

Damage penalty. Where $C_D > 0$: Eq. (88) (`inf_norm`) or Eq. (89) (`quadratic`), for every train and day.

Domains. The variables and their domains are those of Section 5; a variable that no row uses is not built.

The table gives, for each kind of cell, the tracked quantities with their repair factor a (Eq. (10)), their natural ranges M_s (Section 7.4) and the reliability row.

Cell	Tracked quantities (factor a)	Natural ranges M_s	Reliability row
Gaussian	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, $V_{\max}$	Eq. (36), quadratic
Inverse Gaussian	μ $(1-\rho)$	τ	Eq. (44), linear
Wiener	μ $(1-\rho)$, v (1, no anchor)	τ, $V_{\max}$	Eq. (36), quadratic
Gamma	μ $(1-\rho)$	τ	Eq. (62), linear
Rainflow, Markov	μ $(1-\rho)$	τ	Eq. (81), linear
Rainflow, Cantelli	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, V^{C}	Eq. (82), quadratic
Rainflow, Hoeffding	μ $(1-\rho)$, R $((1-\rho)^2)$	τ, $2\tau^2/\kappa$	Eq. (83), quadratic
Rainflow, Bernstein	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, V^{B}	Eq. (85), quadratic
Rainflow, Chernoff	μ $(1-\rho)$, K $(1-\rho)$	τ, $s\,\tau - \kappa$	Eq. (86), linear

Problem class. The problem is a mixed-integer linear program when every quadratic reliability row uses the `tangent` or `pwl` form and the damage penalty is `inf_norm` or absent. A quadratic row in the `exact` form makes it a nonconvex mixed-integer quadratically constrained program, which Gurobi solves with `NonConvex` = 2. The `quadratic` damage penalty alone makes it a convex one (Section 7.13).

Output Specification

The function `solve()` returns one result dictionary for the horizon H of the input, and writes it to a file if `results_path` is not `None`. Array values are `numpy.ndarray` objects; the result holds no Gurobi object. In the column *Shape*, a dash marks a scalar or a string.

Result Dictionary

Key	Shape	Description
Status and quality		
`status`	—	`optimal`, `time_limit`, `suboptimal`, `interrupted`, `solution_limit`, `node_limit`, `iteration_limit`, `infeasible`, `inf_or_unbounded`, `unbounded`, or `gurobi_status_`n for any other Gurobi status code n
`objective`, `mip_gap`, `bound`	— or `null`	Objective value of the best solution found; relative MIP gap reached; best bound, if available
Problem		
`F`, `M`, `L`, `H1`, `H2`, `T`	—	F, M, L, H_1, H_2 and $T = H_1 + H_2$
`H`	integer or $[H_1, H_2]$	Horizon as written in the input
`model`	$F \times L$ list of strings	Degradation model of each cell, as in the input
`component_names`	list of L strings	Names of the components
`tau`, `mu_0`, `v_0`	$F \times L$	Threshold, initial mean and initial variance of each cell, after broadcasting
`bound_method`, `repair_model`, `reliability_impl`, `encoding`	string, or $F \times L$ list of strings	Selector of each cell after broadcasting: one string if all cells agree, else a list; `bound_method` is `null` at a cell that is not rainflow
`objective_mode`, `evaluation_horizon`	—	Objective mode, and E if given
Solution (`null` if no solution was found)		
`x`	$F \times (M{+}1) \times T$	Assignment x_{ijk} (Section 5)
`m`	$F \times L \times T$	Repair $x^m_{i\ell k}$
`r`	$F \times L \times T$	Replacement $x^r_{i\ell k}$; zero if `allow_replacement` is `false`
`idle`	$F \times L \times T$	$\text{idle}_{i\ell k} = x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k}$: component ℓ is in the depot without an intervention
`mu`	$F \times L \times T$	Tracked mean degradation
`v`	$F \times L \times T$	Tracked variance; `NaN` at cells that track no variance (IG, Ga, RF with Mk, Ho or Ch)
`z`	$F \times L \times T$	Repair-cost variable; `NaN` at cells with $C_{R,i\ell} = 0$. On the transitory days of the mode `operating_average`: Equation (91)
`R`, `K`	$F \times L \times T$	Tracked Hoeffding budget (cells RF (Ho)) and Chernoff quantity (cells RF (Ch)); `NaN` elsewhere
`g_mu`, `g_v`, `g_R`, `g_K`	$F \times L \times T$	Tracked anchors $g^\mu_{i\ell k}$, $g^v_{i\ell k}$, $g^R_{i\ell k}$, $g^K_{i\ell k}$ (Section 7.3), only at `ard1` cells: `g_mu` at every model, `g_v` at G and RF (Ca, Be), `g_R` at RF (Ho), `g_K` at RF (Ch); `NaN` elsewhere
`u`	—	Damage penalty value u, Equation (92)
Cost report (`null` if no solution was found)		
`step_costs`	T	Cost c_k of each day, Equation (2)
`J_initialization`, `J_op`, `J_op_average`	—	J_{init}, J_{op} and J_{op}/H_2, Equation (3)

Key	Shape	Description
`peak_damage`, `damage_penalty`	—	u and $C_D\,u$
`operating_objective`, `J_total`	—	$J_{\text{op}}/H_2 + C_D\,u$ and $J_{\text{init}} + J_{\text{op}} + C_D\,u$
`projected_evaluation_cost`, `evaluation_objective`	— or `null`	P_E (Equation (93)) and $P_E + C_D\,u$; `null` if `evaluation_horizon` is not given
`component_costs`	mapping	Cost coefficients used: `C_M`, `C_D` (scalars), `C_R`, `C_rep` ($F \times L$)
Reports (always present)		
`performance`	mapping	Sizes and times of the solve
`optimization_progress`	list of mappings	Progress of the incumbent and the bound
`warm_start`, `relaxation_warm_start`	mappings	What each warm start did

Rules for the Values

If Gurobi found no solution, the solution arrays and the cost report are `null`, and so are `objective` and `mip_gap`; if it found one, the arrays hold the best solution and `status` tells how the solve ended. The arrays `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K` and `z` hold the tracked variables of the model, and are upper bounds (Section 7.4).

In the mode `operating_average`, z is not built on the days of the transitory phase (Section 7.12); at a cell with $C_{R,i\ell} > 0$ the result holds, for those days, the value computed after the solve:

$$z_{i\ell k} = \begin{cases} \rho_{i\ell}\left(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1}\right) & \text{if } x^{m}_{i\ell k} = 1, \\ 0 & \text{otherwise,} \end{cases} \qquad k = 1, \ldots, H_1, \tag{91}$$

with $\mu_{i\ell 0} = \mu_{0,i\ell}$, $g^{\mu}_{i\ell 0}$ the starting anchor, and $g^{\mu} \equiv 0$ under `ardinf`.

The value u is recomputed from `mu`, and is reported also when $C_D = 0$:

$$u = \max_{i,k} \sum_{\ell=1}^{L} \mu_{i\ell k} \quad (\texttt{inf_norm}), \qquad u = \max_{i,k} \sum_{\ell=1}^{L} \mu^2_{i\ell k} \quad (\texttt{quadratic}). \tag{92}$$

The cost c_k of day k and the phase costs J_{init}, J_{op} are Equations (2) and (3) (Section 6) on the arrays of the result, with $z_{i\ell k} = 0$ where it is `NaN`. When the variable u has no slack, `objective` equals, up to the tolerance of the solver, `J_total`, `operating_objective` or `evaluation_objective` for the mode `total`, `operating_average` or `evaluation_total`; otherwise it is larger by C_D times the slack.

Reports

`performance` is a mapping; an entry that Gurobi does not give is `null`. *Keys:*

- size entries: `gurobi_version`; `variables`, with `continuous_variables`, `integer_variables` and `binary_variables`; the rows `linear_constraints`, `general_constraints`, `indicator_constraints` and `quadratic_constraints`; `nonzeros`;
- work entries: `solutions_found` and `gurobi_runtime_seconds`; the counts `branch_and_bound_nodes`, `simplex_iterations` and `barrier_iterations`; `work_units`;

- wall-clock entries, in seconds: `model_construction_seconds`, `optimizer_call_seconds`, `solution_extraction_seconds` and `backend_wall_seconds`;
- settings: `requested_time_limit_seconds`, `requested_mip_gap`, `threads` and `seed`.

`optimization_progress` is a list of records with the entries `runtime_seconds`, `incumbent`, `best_bound`, `relative_gap`, `nodes` and `solutions`; the incumbent, the bound and the gap are `null` while Gurobi has none. A record is made at the first progress report of Gurobi, then at least `progress_interval_seconds` after the previous record and whenever the number of solutions changes; the last record is the final state of the solve.

`warm_start` (Section 11) is a mapping with the entries `applied` (`true` when at least one value was set), `values` (number of values set), `mode` (`"start"`, `"hint"` or `"both"`, `null` without a source), `source_horizon` (the pair H_1, H_2 of the source, or `null`) and `completion`. `completion` (Section 11.3) is a mapping with `requested`, `status`, `fixed_integer_variables`, `values_copied`, `runtime_seconds` and `iis_constraints`. Its `status` is one of `"not_requested"`, `"completed"`, `"infeasible"`, `"infeasible_or_unbounded"` and `"time_limit"`, or `"status_`q`"` for another Gurobi status code q. `iis_constraints` is a list of at most 50 names of rows of an irreducible infeasible subset, empty unless the status is `"infeasible"`.

`relaxation_warm_start` (Section 11.4) is a mapping with the entries `enabled` (the value of the option); `status` (`"disabled"` if the option is false, `"skipped"` if it is true and a start or a hint was applied, otherwise the Gurobi status of the relaxed copy as a string); `relaxation_objective`; `values_copied`; `runtime_seconds`.

Result Files

`solve` writes the result to the file `results_path`.

- The default of `results_path` is `"output.yaml"`; `None` writes no file.
- A path without extension gets the extension `.yaml`. `.yaml` and `.yml` give YAML, `.json` gives JSON, `.h5` and `.hdf5` give HDF5; another extension raises a `ValueError`.
- The extension and the directory of the path are checked before the input file is read: the directory must exist (`FileNotFoundError`) and be writable (`PermissionError`).
- YAML and JSON files hold plain data (lists, numbers, strings, `null`); `NaN` is written as `null`. They load with `yaml.safe_load` and `json.load`. HDF5 files keep `NaN` (Section 14.2).
- A key whose value is `None` is not written, except `objective`, written as `null`. Every reader of a result treats a missing key as `null`.

Horizon Sweeps

A horizon sweep solves one input for several pairs (H_1, H_2) and ranks the cases. `sweep_operating_horizons` varies H_2 for one H_1; `sweep_horizon_grid` varies H_1 and H_2. No input key starts a sweep, and `solve()` never runs one.

Cost Measures and Ranking

J_{init} and J_{op} of a case are Eq. (3) on its solution. The cost per operating day, and the projected cost over an evaluation horizon $E > H_1$, are

$$\bar{J}_{\text{op}} = J_{\text{op}}/H_2, \qquad P_E = J_{\text{init}} + (E - H_1)\,\bar{J}_{\text{op}}. \tag{93}$$

A case is *feasible* if its result holds a solution and its status is not `"infeasible"`, `"unbounded"` or `"inf_or_unbounded"`. A case is *proven* if it is feasible and its status is `"optimal"` at the

MIP gap used for the case: Gurobi's `MIPGap` after all options are set, `gurobi_params` included (Section 4.10); it is the entry `requested_mip_gap` of `performance`. The *ranking value* is $\bar{J}_{\text{op}}$ in `sweep_operating_horizons`; in `sweep_horizon_grid` it is P_E if an evaluation horizon is given, otherwise $\bar{J}_{\text{op}}$. The *best proven case* and the *best feasible case* have the smallest ranking value among the proven and the feasible cases. A tie goes to the smaller H_2; in the grid, to the smaller H_1, then the smaller H_2.

`sweep_operating_horizons`

```
sweep_operating_horizons(
    input_path, operating_horizons, *, output_path=None,
    stop_on_gradient=False, gradient_tolerance=1e-3,
    flat_gradients_required=2, minimum_cases=3,
    maximum_mip_gap_for_stopping=None,
    warm_start=False, warm_start_mode="start", n_workers=1,
    objective_mode="operating_average", evaluation_horizon=None) -> dict
```

Arguments with a rule:

- `input_path`: H_1 is the first entry of `H`, or `H` if it is a scalar; the second entry is ignored.
- `operating_horizons`: the values of H_2; positive integers, at least one, no duplicates; evaluated in increasing order.
- `output_path`: the report file, `.yaml` or `.yml`; missing parent folders are created; `None` writes no file.
- The arguments of the early stop (Section 10.3): `gradient_tolerance` is finite and ≥ 0; `flat_gradients_required` is ≥ 1; `minimum_cases` is ≥ 2; `maximum_mip_gap_for_stopping` is `None` or a finite number ≥ 0.
- `objective_mode`, `evaluation_horizon`: written into every case input; `"evaluation_total"` needs $E > H_1$. For `warm_start`, `warm_start_mode` and `n_workers`, see Sections 11.5 and 10.5.

Steps:

1. Read the input file with the input reader (Section 14); the data must be a mapping (`TypeError`). Check every argument.
2. For each H_2, the case input is the input data with `H` $= [H_1, H_2]$, and with `objective_mode` and `evaluation_horizon` (if given) replaced. Time-indexed data are not interpolated: a profile whose length is not H_2 fails the checks of Section 4.12, as an error of that case.
3. Solve the case input with `solve_mapping` (Section 2.2) and `results_path=None`; no temporary file is written. Keep the row of the case, not its schedule.
4. After each case: append its row, annotate the gradient (Section 10.3), test the stop rule, write a checkpoint if `output_path` is given. At the end, write the final report and return it.

Errors. `ValueError` before the first case for an invalid argument, for a list `H` whose length is not two, and for `warm_start` or `stop_on_gradient` with `n_workers` > 1; such an error is never recorded as a case error. An error inside a case is captured in its row (Section 10.4).

Early Stop

Let $\bar{J}_n$ be the cost per operating day of the n-th evaluated case. If cases $n-1$ and n are feasible, the relative gradient is

$$\hat{g}_n = \frac{\bar{J}_n - \bar{J}_{n-1}}{(H_{2,n} - H_{2,n-1}) \max\{\, |\bar{J}_{n-1}|,\ \varepsilon_{\text{mach}} \}}, \tag{94}$$

with $\varepsilon_{\text{mach}}$ the machine precision of double-precision numbers. With $t =$ `gradient_tolerance`, the *class* of case n is `"increase"` if $\hat{g}_n > t$, `"decrease"` if $\hat{g}_n < -t$, `"flat"` otherwise, and `"unavailable"` if $n = 1$ or one of the two cases is not feasible. A case *qualifies* if it reports a finite MIP gap at most its threshold: `maximum_mip_gap_for_stopping` if it is a number, otherwise the MIP gap used for the case (`requested_mip_gap` of its row). The gradient of case n is *eligible to stop* if cases $n-1$ and n qualify. Every row carries its gradient, class and eligibility, also if `stop_on_gradient` is false. With `stop_on_gradient=True` the sweep stops after case n if $n \geq$ `minimum_cases`, the gradient of case n is eligible, and either the class of case n is `"increase"` (reason `"cost_increase"`) or the last `flat_gradients_required` cases up to n all have the class `"flat"` and an eligible gradient (reason `"flat_gradient"`). The last candidate of `operating_horizons` is a hard limit.

Report, Checkpoint and Errors

The report is a mapping of plain Python types, written as YAML. *Keys:* `input` (input path); `complete` (`false` in a checkpoint); `last_completed_H2`; `objective` (a text); `varied_parameter` (`"H2"`); `fixed_dimensions` (F, M, L) and `H1`; `planned_operating_horizons` and `evaluated_operating_horizons`; `warm_start` (`enabled`, `mode`) and `n_workers`; `stopping_rule`, with `enabled`, `relative_gradient_tolerance_per_H2`, the stop arguments `flat_gradients_required`, `minimum_cases` and `maximum_mip_gap_for_stopping` under their own names (the last one `null` for the default), the `reason` (`"in_progress"`, `"candidate_range_exhausted"`, `"cost_increase"` or `"flat_gradient"`) and `stopped_early` (complete with fewer cases than planned); `cases` (the rows, by H_2); `formulation_growth` from the first to the last row with counts: `from_H2`, `to_H2`, `added_time_steps` and, for each count q of `formulation`, `added_`q and q`_per_added_time_step` (`null` with fewer than two such rows); for $b \in$ {`proven`, `feasible`}, `best_`b`_H2` and `best_`b`_J_op_average`, and `best_feasible_status` (`null` without such a case).

Keys of a case row: `H1`, `H2`, `T`, `dimensions` (F, M, L, H_1, H_2, T); from the result of the case (Section 9), `status`, `objective_mode`, `evaluation_horizon`, `objective`, the scalar keys of the cost report, and the warm-start report `warm_start`; `objective_bound`, `mip_gap` and `absolute_mip_gap` $= \max\{0,\ \text{objective} - \text{bound}\}$; `requested_mip_gap` and `requested_time_limit_seconds`, Gurobi's `MIPGap` and `TimeLimit` after all options are set; `optimizer_seconds`, `solutions_found`; `formulation`, the size entries of `performance` from `variables` to `nonzeros`; `timing`, the wall-clock entries of `performance` with `branch_and_bound_nodes`, `simplex_iterations` and `work_units`; the gradient keys `cost_gradient_per_H2`, `relative_cost_gradient_per_H2`, `gradient_classification` and `gradient_stopping_eligible` (Section 10.3).

Checkpoint. With `output_path`, the report is rewritten after every case, with `complete = false` and the reason `"in_progress"` until the sweep ends.

Case errors. An exception (class `Exception`) raised by a case after the arguments are accepted is logged at level ERROR with H_1 and H_2. The row of the case has the status `"error: "` followed by the message, and `null` in every entry except the sizes; the sweep continues. Such a row is neither feasible nor proven, and is never a warm-start source.

Parallel Cases

With `n_workers` > 1, up to `n_workers` cases are solved at the same time, each in its own process with its own Gurobi environment and one Gurobi licence token. The option is refused with `warm_start` and with `stop_on_gradient`. Rows arrive in the order of completion; the report orders them by H_2, and each rewrite recomputes the gradient annotations over the ordered rows.

`sweep_horizon_grid`

```
sweep_horizon_grid(
    input_path, transitory_horizons, operating_horizons, *,
    output_path=None, warm_start=True, evaluation_horizon=None,
    **sweep_options) -> dict
```

Both horizon lists hold positive integers; they are sorted and repeated values are removed. `evaluation_horizon`, if given, must exceed every H_1 (`ValueError`). `sweep_options` are the keyword arguments of `sweep_operating_horizons` except `objective_mode`. With `n_workers` $>$ 1, `warm_start=False` must be passed.

For each H_1 in increasing order, `sweep_operating_horizons` runs on the input data with that H_1 and the list of H_2, with `objective_mode "evaluation_total"` if an evaluation horizon is given and `"operating_average"` otherwise. Warm starts (Section 11.5), the early stop and the capture of case errors act within each H_1; the early stop uses $\bar{J}_{\text{op}}$ also when E is given. The grid report is rewritten after every case (`complete = false`) and at the end.

Keys: `input`, `complete` and `objective` as in Section 10.4; `comparison_metric`, the ranking key (`"projected_evaluation_cost"` or `"J_op_average"`); `mip_certificate_metric` (`"objective"`); `evaluation_horizon`; `varied_parameters` (`["H1", "H2"]`); `fixed_dimensions` (F, M, L); the candidates `planned_transitory_horizons` and `planned_operating_horizons`; the flag `warm_start_enabled`; `H1_sweeps`, the report of each operating sweep, with the input file of the grid as `input`; `cases`, all rows by H_1, then H_2; `best_proven` and `best_feasible`, `null` or a mapping with `H1`, `H2`, `T`, `status`, `mip_gap`, `objective` and the scalar keys of the cost report except `J_op` and `J_total`.

Warm Starts

A warm start gives Gurobi values for the binary variables. Four mechanisms exist: a *start* (attribute `Start`) from a source, with an optional completion; *hints* (attribute `VarHintVal`) from a source; a *relaxation start*; and the *greedy heuristic*, which builds a source from the input.

Interface and Reports

`solve()` takes `warm_start`, a source or `None` (default), and `warm_start_mode`: `"start"` (default), `"hint"` or `"both"`; another value raises `ValueError`. A *source* is a result dictionary or the dictionary returned by the greedy heuristic (Section 11.6). It carries `x` (shape $F' \times (M'+1) \times T'$) and the integers `H1`, `H2` with $T' = H_1' + H_2'$; it may carry `m` and `r` (shape $F' \times L' \times T'$). A source whose `x` is missing or `null` has no effect. F', M', L' may differ from F, M, L: values are set for the indices that both have. The result keys `warm_start` and `relaxation_warm_start` report what each mechanism did (Section 9.3).

Day Mapping

Day $k \in \{1, \dots, T\}$ takes the values of the source day

$$\sigma(k) = \begin{cases} k, & k \le \min\{H_1, H_1'\}, \\ H_1' + 1 + \big(\max\{0,\ k - H_1 - 1\} \bmod H_2'\big), & \text{otherwise.} \end{cases} \tag{95}$$

The mapped values, rounded to the nearest integer, are $x_{ijk} \leftarrow x'_{ij\sigma(k)}$ for $j = 0, \dots, M$; $x^m_{i\ell k} \leftarrow m'_{i\ell\sigma(k)}$, or 0 if the source has no `m`; and $x^r_{i\ell k} \leftarrow r'_{i\ell\sigma(k)}$ if `allow_replacement` is true and the source has `r`. A binary variable that these values fix through its defining rows (the selector $n_{i\ell k}$ of Eq. (16)) gets that value.

Start, Hints and Completion

Mode `"start"` sets `Start` of every mapped variable; continuous variables get no value. Mode `"hint"` sets `VarHintVal` instead, with no `Start`, no completion and no `VarHintPri`. Mode `"both"` sets `Start` and `VarHintVal`. In the modes `"start"` and `"both"` the completion runs if the source requests it.

Completion. Only the source requests it, with `complete_binary_start = true`; the greedy heuristic does. It runs if the mapping set at least one value, with the time limit `completion_time_limit` of the source (seconds, default 60).

1. Copy the model; fix every binary variable that has a `Start` value to that value; the other binary variables stay free. Optimise the copy within the time limit.
2. If the copy has a solution, set `Start` of every variable of the original model to its value in the copy.
3. Otherwise record the status of the copy. If it is infeasible, compute an irreducible infeasible subset (IIS, Gurobi `computeIIS`) of the copy and record the names of at most 50 of its linear rows and general constraints.

The original model is not changed; a failed completion does not stop the solve.

Relaxation Start

If the run option `relaxation_warm_start` (Section 4.10) is true and no start or hint was applied (`applied` is false in the report `warm_start`):

1. Copy the model and relax its linear rows with Gurobi's `feasRelaxS` (sum of absolute violations); integrality, variable bounds and general constraints are kept.
2. Optimise the copy with the time limit $\max\{1,$ `relaxation_time_limit`$\}$ seconds and the MIP gap 0.05.
3. If the copy has a solution, set `Start` of every integer variable of the original model to the rounded value of the variable of the same name in the copy.

The relaxed solution is not a point of the original model.

Warm Starts in Sweeps

With `warm_start=True`, the source of a case is the result of the nearest preceding case of the same H_1 that has a solution, used in the mode `warm_start_mode`. The first case of each H_1 has no source; a case with an error or without a solution is never a source.

Greedy Heuristic

```
build_greedy_warm_start(cfg, *, repair_trigger=0.70)
    -> (dict, GreedyWarmStartReport)
```

Module `fleet_management.greedy_warm_start`, not exported; it works for every model. `cfg` is the normalised configuration (`load_config`, Section 2.2); $r =$ `repair_trigger` must lie in $[0, 1]$ (`ValueError`). The returned source has the keys `H1`, `H2`, `x`, `m`, `r`, `complete_binary_start` (`true`) and `completion_time_limit` (60).

Construction pass. A proxy state $\hat{s}_{i\ell}$ starts at $\mu_{0,i\ell}$. For each day $k = 1, \ldots, T$:

1. Order the missions by decreasing $\max_{i,\ell} \mu^{\text{inc}}_{i\ell jk}/\tau_{i\ell}$; a tie goes to the larger j.
2. In this order, give each mission to the train i without a mission on day k that minimises, in lexicographic order, $\big(\max_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell},\ \sum_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell}\big)$; a tie goes to the smaller i. Add $\mu^{\text{inc}}_{i\ell jk}$ to $\hat{s}_{i\ell}$. The check $F > M$ (Section 4.12) guarantees a train for every mission.

3. A train without a mission is idle, unless some component has $\hat{s}_{i\ell} > r\,\tau_{i\ell}$: then $x_{i0k} = 1$, and every such component is repaired ($x^m_{i\ell k} = 1$, $\hat{s}_{i\ell} \leftarrow (1 - \rho_{i\ell})\,\hat{s}_{i\ell}$ for every repair law).

The construction pass schedules no replacement.

Loop pass. For each cell, compute the reference trajectory ζ (Section 7.4) of the current schedule from the initial values, starting anchors and replacement values of the inputs. The excess e of the cell is the largest of the loop excesses $\max\{0,\ \zeta^q_T - \zeta^q_{H_1}\}/M_q$ of every tracked quantity and anchor q with natural range M_q (Section 7.14), and of the violations of the reliability row on every day, divided by the natural range of the bounded quantity. The row is taken in the form built at the cell (Sections 7.7–7.11): the exact row for `"exact"`, the tangent row for `"tangent"`, and for `"pwl"` the cap of the segment that contains the mean of the day (Section 7.6). While $e > 10^{-10}$: for every free day $k \in \{H_1 + 1, \dots, T\}$ (the train has no mission, the cell no intervention), put a repair of the cell on day k (the train goes to the depot) and recompute e; keep the repair with the smallest e, a tie to the later day. If no repair lowers e by more than 10^{-10} and replacement is allowed, repeat the trial with a replacement. If no trial lowers e, count one failure for the cell and go to the next cell. A start with failures is returned.

Report. `GreedyWarmStartReport.as_dict()` returns plain Python types. Keys: `repairs` and `replacements`, the numbers of ones in `m` and `r`; `repeatability_feasible`, true without a failure; `repeatability_failures`, the number of cells with a failure; `maximum_repeatability_excess`, the largest raw excess over all cells after the loop pass; `maximum_proxy_ratio`, the largest $\hat{s}_{i\ell}/\tau_{i\ell}$ of the construction pass and reference mean over $\tau_{i\ell}$ of the loop pass; `assignment_order`, for each day, the missions in the order treated.

Plotting API and Layout

Both plots are in `utils/plotter.py`.

Schedule Plot

```
plot_management(input_file_path, plot_file_path=None) -> None
```

`input_file_path` is a result file of `solve()` (YAML, JSON or HDF5). `plot_file_path` is a `.png` or `.pdf` image, default `"output.png"`; a path without extension gets `.png`. Errors: `FileNotFoundError` for a missing result file or image folder; `PermissionError` for an image folder that is not writable; `ValueError` for another extension of either file. Any result with a solution is plotted, whatever its status. `ValueError`, naming the keys, if the result lacks one of `F`, `M`, `L`, `H1`, `H2`, `tau`, `mu_0`, `mu`, `x`; if a shape differs from Section 9; if $H_1 + H_2 \neq T$; or if `component_names` has a length other than L. Missing `m` and `r` are read as zero; the default name of component ℓ is "Component ℓ" (ℓ from 1); the model of each row is read from the key `model`.

Layout. A grid on the left, a summary panel on the right.

- Rows: one per cell (i, ℓ), ordered by train, then component. Each label has two lines, "Train i — *component name*" and the model name. A heavier line separates the trains.
- Columns: "initial" ($\mu_{0,i\ell}$) and the days $1, \dots, T$; a vertical dashed line after day H_1; the labels "transitory" and "operating" above the grid.
- Colour: a green-to-red scale of $\mu_{i\ell k}/\tau_{i\ell}$, clipped to $[0, 1]$; colour bar "tracked mean / failure threshold".
- Marks: one per field of a day, given by the first rule of the table that applies; none in the column "initial". A value counts as 1 if it is above 0.5. A legend names the five marks.

Mark	Rule for cell (i, ℓ) on day k
`P`	replacement: $x^r_{i\ell k} = 1$
`R`	repair: $x^m_{i\ell k} = 1$
`M`$_j$	train i flies mission $j \geq 1$: $x_{ijk} = 1$ (the mark shows j)
`D`	depot day without intervention: $x_{i0k} = 1$, $x^m_{i\ell k} = x^r_{i\ell k} = 0$
`I`	idle day of the train (Section 7.1)

The summary panel "Run summary" shows the values that the result holds: status (coloured by outcome), objective, sizes and horizon, models and repair laws present, objective mode with J_{op}/H_2, MIP gap in percent, numbers of variables (all, continuous, integer) and linear rows, optimizer time, branch-and-bound nodes, and the numbers of repairs and replacements. The counts come from `performance`; if a Gurobi log `runner.out` lies in the folder of the result file, the panel shows instead the presolved counts of its lines "Presolved: r rows, c columns" and "Variable types: a continuous, b integer", and says so.

Sweep Plot

```
plot_horizon_sweep(sweep_report_path, plot_file_path=None) -> None
```

`sweep_report_path` is one path, or a list of paths, of reports of `sweep_operating_horizons` (YAML or JSON); `ValueError` for another file type or an empty list, `FileNotFoundError` for a missing file. `plot_file_path`: as for `plot_management`, with the default `"horizon_sweep.png"`. The reports must agree in F, M, L, H_1, and each needs a list `cases` (`ValueError`). For equal H_2 the plot keeps the case with a solution, then the smaller J_{op}/H_2, then the larger objective bound. The title names the sweep and F, M, L, H_1. Three panels over H_2 show: (a) J_{op}/H_2 of each case, with one mark for a proven case (status `"optimal"`), one for a feasible case that is not proven, and one for a case without a feasible solution (error rows included), a mark on the best feasible case, and the values of the proven cases; (b) the relative MIP gap in percent, with a horizontal line at `maximum_mip_gap_for_stopping` of `stopping_rule` if it is a number, otherwise a mark at the requested MIP gap of each case; (c) the numbers of continuous variables, integer variables and linear constraints.

Validation Tools

The tools check a saved result against its input; they do not call the optimiser. Each tool reads the input with the input reader and `load_config` (Section 2.2), reads a result file in any format that `solve()` writes, and takes the model of each cell from the result key `model`.

Structural Validator

```
validate(input_path, results_path, validation_path="validation.yaml",
         tol=1e-6) -> dict
```

The report is written to `validation_path` (`.yaml`, `.yml` or `.json`; no extension gets `.yaml`; the folder is created) and returned. Errors: `FileNotFoundError` for a missing file; `PermissionError` for a folder that is not writable; `ValueError` for an unsupported extension, a result without `x` (missing or `null`), an input that fails Section 4.12, or sizes F, M, L, H_1, H_2 or array shapes that differ between result and input (named in the message). The status is not a check: a feasible incumbent is validated like an optimal solution, and the report echoes the status. A check passes if its violation is at most `tol`. The checks, each with its violation:

- `binary`: x, x^m, x^r take only the values 0 and 1; the largest distance of an entry from $\{0, 1\}$.

- `one_activity`: $\sum_{j=0}^{M} x_{ijk} \le 1$ (Section 7.1); $\max\{0,\ \max_{i,k}(\sum_j x_{ijk} - 1)\}$.
- `mission_coverage`: every mission $j = 1, \dots, M$ has $\sum_i x_{ijk} = 1$ (the depot $j = 0$ is not summed); $\max_{j \ge 1,k} |\sum_i x_{ijk} - 1|$.
- `depot_gating`: $x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k}$ (Section 7.2), and $x^r = 0$ if `allow_replacement` is false; the largest excess of the left side over the right side.
- `idle`, if the result has the key `idle` (Section 9): $\max |\texttt{idle}_{i\ell k} - (x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k})|$.
- `loop_`q for $q \in \{\texttt{mu}, \texttt{v}, \texttt{R}, \texttt{K}, \texttt{g_mu}, \texttt{g_v}, \texttt{g_R}, \texttt{g_K}\}$: the loop row $q_{i\ell T} \le q_{i\ell H_1}$ (Section 7.14), with `NaN` and `null` entries skipped; $\max\{0,\ \max_{i,\ell}(q_{i\ell T} - q_{i\ell H_1})\}$.
- `objective`: the saved objective o is not below the recomputed objective r; $\max\{0,\ r - o\} / \max\{1, |o|\}$. The recomputed objective is Eq. (4) in the objective mode of the input, with the day cost of Eq. (2) on the saved x, z (0 where `NaN`) and x^r, the costs of the input, and the saved u.

Keys: `status`, `solver_objective`, `recomputed_objective`, `tolerance`; `checks`, a list of records with `name`, `passed`, `violation` and `tolerance`; `passed`, true if all checks passed; `max_violation`.

Dashboards

Streamlit pages (extra `dashboard`), started with

```
python -m streamlit run src/fleet_management/validator_dashboard.py
```

The sidebar selects the mode, the input file, the result file and the tolerance (default 10^{-6}); a button loads the pair. The dashboards read only these two files. A failed load shows the error and no page.

Mode "Result inspector" (every model). Tabs: *Overview* (status, objective, models, H_1, H_2, T, number of reliability violations, largest μ/τ, cells with the smallest margin); *Reliability* (margins by train, component and day; filters for train, component and violations only); *Repeatability* (the loop row of each cell and saved tracked quantity, Section 7.14); *Train timeline* (the means of one train with τ and the marks of Section 12); *Heatmap* (margins over cells and days); *Component comparison* (one component across the trains); *Validation* (the checks of `validate`); *Raw data* (with an export). The *margin* of a cell on a day is the right side minus the left side of the exact form of its reliability row (Sections 7.7–7.11), on the saved tracked states. The row holds if the margin is at least $-$`tolerance`; `NaN` entries are left out.

Mode "Gamma diagnostic" (Gamma cells), which runs the replay and the exact validators (Section 13.3). Tabs: an overview (repairs, replacements, outcome of the replay validator); the tracked mean of each Gamma cell with its cap $A_1(\beta^*\tau, \varepsilon)/\beta^*$ (Section 7.10); the exact tail probability of the true damage against ε, also as a heatmap; raw data. The mode needs no schedule inside the input file.

Validators of the Gamma Model

Each validator takes the normalised configuration `cfg` (`load_config`) and the result dictionary, and returns a report of plain Python types. Its file version takes an input path, a result path and `report_path` (default `None`); it writes the report as YAML or JSON (`ValueError` for another suffix) and creates the folder. `ValueError` if `cfg` has no Gamma cell; if a required array is missing, has a wrong shape or a non-finite entry; if an array that must be binary is not within `tolerance` of $\{0, 1\}$. `tolerance` (default 10^{-8}) must be positive. With `raise_on_failure=True`, a report with `valid = false` raises `AssertionError`.

Replay validator.

```
validate_gamma_replay_schedule(cfg, result, *, tolerance=1e-8,
```

```
                                raise_on_failure=False) -> dict
validate_gamma_replay_files(input_path, result_path, report_path=None,
                            **kwargs) -> dict
```

It reads `x`, `m`, `r` (optional), `mu`, `z` and, for ARD_1 cells, `g_mu`. Global violations (strings in `global_violations`): a result horizon other than T; a train with more than one activity; a mission not covered exactly once; a repair and a replacement of a cell on one day; an intervention on a day with a mission or without a depot day. For each Gamma cell and day k, the reference trajectory of μ and g^{μ} (Section 7.4) is computed from the saved schedule, with the initial value, starting anchor and replacement value of the cell. The event of the day is `"replacement"`, `"repair"`, `"mission"` or `"idle"`. The errors are:

- `tracked_mean`: $\max\{0,$ reference mean $-$ $\mathtt{mu}_{i\ell k}\}$; `mean_anchor`: the same for `g_mu`, under ARD_1;
- `removed_mean`: $\max\{0,\ x^{m}_{i\ell k}\,\rho_{i\ell}(\mu_{i\ell,k-1} - g_{i\ell,k-1}) - z_{i\ell k}\}$, with $g = 0$ under ARD_{∞} (Section 7.12); skipped where z is `NaN` ($C_{R,i\ell} = 0$);
- `cap`: $\max\{0,\ \mathtt{mu}_{i\ell k} - A_1(\beta^{*}\tau, \varepsilon)/\beta^{*}\}$ (Section 7.10);
- `repeatability`: $\max\{0,\ \mathtt{mu}_{i\ell T} - \mathtt{mu}_{i\ell H_1}\}$, and the same for the saved anchor.

An error above `tolerance` adds a record `i`, `l`, `k`, `event`, `field`, `expected`, `saved`, `error` to `violations`. *Keys:* `validator`; `valid`, true without a global violation and without a violation; `solver_status`, `dimensions`, `gamma_cells`, `gamma_ard1_cells`, `transitions_checked`, `repairs`, `replacements`; `maximum_errors`, the largest error of each field; `global_violations`, `violations`, `timing`.

Exact validator (module `gamma_utils.gamma_tail_validator`, not exported).

```
validate_gamma_tail_bound_schedule(cfg, result, *, tolerance=1e-8,
    convolution_tolerance=1e-12, max_series_terms=100000,
    include_steps=False, raise_on_failure=False) -> dict
validate_gamma_tail_bound_files(input_path, result_path, report_path=
   None,
                                **kwargs) -> dict
```

For each Gamma cell and day $k = 1, \ldots, T$, the true damage is a list of independent Gamma terms (Section 7.10). A mission adds a term with the exact rate of its increment. A repair multiplies the rate of the repairable terms by $1/(1-\rho)$, and removes them for $\rho = 1$; under ARD_1 the repaired terms join the frozen part of the anchor. A replacement starts a new list with the replacement term. An initial or replacement damage without a rate (`gamma_beta_0` or `gamma_beta_new` omitted) is a known constant c, not a term: the tail is $\Pr(\text{sum of the terms} > \tau - c)$, and 1 if $c > \tau$. The tail $\Pr(D_k > \tau)$ is computed by the Moschopoulos series (with equal rates, the Gamma tail); the upper bound is the partial sum plus the remaining mixture mass, which is at most `convolution_tolerance`. `RuntimeError` if the series does not reach this tolerance within `max_series_terms` terms. `convolution_tolerance` must lie in $(0, 1)$; `max_series_terms` must be positive. A day passes if the upper bound is at most $\varepsilon_{i\ell} + $ `tolerance`, the saved mean is at least $\mathbb{E}D_k -$ `tolerance` and, under ARD_1, the saved anchor is at least the exact mean of the frozen part minus `tolerance`. *Keys:* those of the replay report; `minimum_reliability_slack`, the minimum of ε minus the upper bound; `worst_reliability`, the step that attains it; `maximum_remaining_mass`; the numbers of series terms and the timing of the tail computations; with `include_steps=True`, the list `steps` (terms and values of every cell and day).

Monte Carlo validator.

```
validate_gamma_stochastic_schedule(cfg, result, *, repetitions=100000,
    random_seed=20260908, maximum_schedule_failure_rate=None,
    confidence_level=0.95, batch_size=20000, tolerance=1e-8,
```

```
    raise_on_failure=False) -> dict
validate_gamma_schedule(cfg, result, *, mode=None, repetitions=None,
    random_seed=None, maximum_schedule_failure_rate=None,
    confidence_level=None, batch_size=None, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule_files(input_path, result_path, report_path=None,
                              **kwargs) -> dict
```

The schedule is fixed, and only Gamma cells are sampled. In each of `repetitions` runs, the true damage of every Gamma cell is drawn: the initial damage, each mission increment and each replacement draw are independent Gamma variables with the means and rates of the inputs. Under ARD_1 the initial damage is the sum of two independent terms with means g_0^{μ} (the anchor) and $\mu_0 - g_0^{\mu}$; a damage without a rate is the constant of its mean. Each day the repair law acts on the sampled path: $D \leftarrow (1-\rho)\,D$ under ARD_{∞}; $D \leftarrow g + (1-\rho)(D-g)$, then $g \leftarrow D$, under ARD_1. A replacement draws the damage of the replacement part and, under ARD_1, sets g to it. After each day the run records whether $D > \tau$. A *schedule failure* is a run in which some Gamma cell exceeds its threshold on some day. The generator is seeded with `random_seed`, and draws are made in batches of `batch_size` runs; the same arguments give the same report. `confidence_level` must lie in $(0, 1)$; `repetitions` and `batch_size` must be positive; `maximum_schedule_failure_rate`, if given, must lie in $[0, 1]$. Each failure rate (of the fleet, of a cell, of the worst day of a cell) comes with its one-sided Clopper–Pearson upper bound at `confidence_level`: for f failures in n runs, the `confidence_level`-quantile of the law $\mathrm{Beta}(f+1,\ n-f)$, and 1 if $f = n$. `valid` is true if no target is given, or if the observed schedule failure rate is at most `maximum_schedule_failure_rate`. *Keys:* `validator`, `mode`, `valid`, `timing`, the arguments used; `scope` ("Gamma cells only"); `complete_fleet_validation` (false if the fleet has other cells); `excluded_cells` (`i`, `l`, `component`, `model`, `reason` of each other cell); `schedule` (`failed_replays`, `failure_rate`, `failure_rate_upper_confidence_bound`, `maximum_failure_rate`, `observed_target_met`, `confidence_qualified_target_met`); `cells`, for each Gamma cell `i`, `l`, `component`, `repair_model`, `threshold`, `analytical_epsilon`, `failed_replays`, `failure_rate` with its bound, `worst_time_step` with its failures, rate and bound, `maximum_sampled_damage`, `maximum_threshold_excess`.

`validate_gamma_schedule` runs the validators that `mode` selects: `"deterministic"` (replay), `"stochastic"` (Monte Carlo) or `"both"`, with the keys `deterministic` and `stochastic` and `valid` true if both are valid; another value raises `ValueError`. An argument that is `None` takes the value of the key of the same name in the input block `validation` (Section 4.10), otherwise its default; the default of `mode` is `"deterministic"`.

Input File Schema

Format Rules

The formats are YAML (`.yaml`, `.yml`), JSON (`.json`) and HDF5 (`.h5`, `.hdf5`), chosen by the extension of the path in lower case; another extension raises an error. One reader serves every function that takes an input path.

- A file holds one mapping from the keys of Section 4 to values, with the same keys and meaning in the three formats.
- An array is a nested list in YAML and JSON; the outermost list runs over the first axis of the shape (train, component, mission, day).
- YAML is read in the safe mode.

HDF5 Layout

An HDF5 file is flat: every key is an attribute or a dataset at the root. The reader makes one entry of every root attribute and root dataset; a byte string is decoded as UTF-8, and a text attribute that begins with `{` or `[` is parsed as JSON.

Input file.

- A scalar number, a flag or a single string is an attribute, or a dataset of shape (); `H` is one integer or two integers.
- An array of numbers is a `float64` dataset with the shape of the key; a `null` entry is `NaN`.
- A selector array and `component_names` are datasets of strings, or JSON text; a `null` entry of a selector is the empty string. `gurobi_params` and `validation` are JSON text.

Output file. A number or a string is an attribute; an array of numbers is a `float64` dataset, with `NaN` kept; a list of strings, a mapping or a list of records is JSON text in an attribute; a key whose value is absent is not stored.

Placement	Keys
Attribute: number or string	`status`, `objective`, `mip_gap`, `bound`, `F`, `M`, `L`, `H` (one or two integers), `H1`, `H2`, `T`, `objective_mode`, `evaluation_horizon`, `u`, `J_initialization`, `J_op`, `J_op_average`, `peak_damage`, `damage_penalty`, `operating_objective`, `J_total`, `projected_evaluation_cost`, `evaluation_objective`; the selectors `bound_method`, `repair_model`, `reliability_impl`, `encoding` when they are one string
Dataset (`float64`)	`x`, `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K`, `z`, `m`, `r`, `idle`, `step_costs`, `tau`, `mu_0`, `v_0`
JSON text in an attribute	`model`, `component_names`, `component_costs`, `performance`, `optimization_progress`, `warm_start`, `relaxation_warm_start`; the selectors `bound_method`, `repair_model`, `reliability_impl`, `encoding` when they differ between cells

Keys by Cell Type

The table lists the keys that each cell type requires, and the optional keys that it reads in addition to those of the first row. `*_trans` is optional for every profile key; `tangent_ref` and `pwl_points` are read only at cells with the form `tangent` or `pwl`; the starting anchors are read only at `ard1` cells.

Cell type	Required	Optional (defaults in Section 4)
Every cell	`F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep`, `C_D`	`L`, `repair_model`, `reliability_impl`, `encoding`, `tangent_ref`, `pwl_points`, `anchor_mu_0`, `replacement_mu`, `replacement_as_new`, `allow_replacement`, `penalty_type`, `component_names`, `mu_trans`, the run options
G	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
IG	`eta`	none
W	`sigma`	`v_0`, `replacement_v`, `v_max_user`
Ga	`gamma_beta`	`gamma_beta_0`, `gamma_beta_new`, `gamma_beta_trans`
RF, Mk	`bound_method` with the value `markov`	none

Cell type	Required	Optional (defaults in Section 4)
RF, Ca	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
RF, Ho	`support`	`R_0`, `R_new`, `anchor_R_0`, `support_trans`
RF, Be	`v`, `support`	`v_0`, `support_0`, `replacement_v`, `support_new`, `anchor_v_0`, `v_max_user`, `v_trans`, `support_trans`
RF, Ch	`cgf`, `s_chernoff`	`cgf_0`, `cgf_new`, `anchor_K_0`, `cgf_trans`

YAML Example (Mixed Models)

Three trains, two missions, two components; the six cells use G, IG, W, Ga, RF with Be and RF with Ch. The example passes every check of Section 4.12.

```
F: 3
M: 2
L: 2
H: [2, 3]                # H1 = 2, H2 = 3, so T = 5
component_names: [Wheelset, Brake pad]

model: [[gaussian, inverse_gaussian], [wiener, gamma], [rainflow,
   rainflow]]
repair_model: [[ard1, ard1], [ardinf, ard1], [ard1, ard1]]
bound_method: [[null, null], [null, null], [bernstein, chernoff]]
reliability_impl: [[tangent, null], [pwl, null], [exact, null]]
encoding: [[bigm, bigm], [bigm, bigm], [bigm, indicator]]

tau: 1.0
epsilon: [[0.05, 0.05], [0.05, 0.05], [0.10, 0.10]]
tangent_ref: 0.4         # tangent cell only
pwl_points: 6            # pwl cell only
rho: [0.8, 0.7]          # per component, the same for every train

mu_0: [[0.12, 0.10], [0.15, 0.08], [0.09, 0.14]]
v_0: [[0.0004, null], [0.0005, null], [0.0004, null]]
support_0: [[null, null], [null, null], [0.05, null]]
anchor_mu_0: [[0.10, 0.06], [null, 0.05], [0.00, 0.10]]
anchor_v_0: [[0.0002, null], [null, null], [0.0002, null]]
anchor_K_0: [[null, null], [null, null], [null, 0.50]]

replacement_mu: 0.02
replacement_v: [[0.00004, null], [0.00004, null], [0.00004, null]]
support_new: [[null, null], [null, null], [0.03, null]]
replacement_as_new: false
allow_replacement: true

mu: [[0.06, 0.05], [0.04, 0.07]]         # profile (L, M)
mu_trans: [[[0.07, 0.06], [0.06, 0.05]], [[0.04, 0.04], [0.07, 0.07]]]
   # (L, M, H1)
v: [[0.0001, 0.0001], [null, null]]
support: [[0.05, 0.04], [null, null]]
cgf: [[null, null], [0.22, 0.38]]        # at s_chernoff = 5
s_chernoff: [[null, null], [null, null], [null, 5.0]]

sigma: [[null, null], [0.03, null], [null, null]]
eta: [[null, 0.15], [null, null], [null, null]]
v_max_user: [[null, null], [0.05, null], [null, null]]
```

```
gamma_beta: [[null, null], [null, 20.0], [null, null]]
gamma_beta_new: [[null, null], [null, 20.0], [null, null]]  # no
    gamma_beta_0

C_M: 1.0
C_R: [2.0, 1.5]
C_rep: [10.0, 8.0]
C_D: 1.5
penalty_type: inf_norm
objective_mode: evaluation_total
evaluation_horizon: 30

verbose: 0
mip_gap: 0.05
time_limit: 600
fast: false
gurobi_params: {Threads: 4}
progress_interval_seconds: 10
relaxation_warm_start: false
```